SCOTCH 7.0.16 bug review

Repository `/data/Tools/scotch` at commit `b748bc4` · Oct 7, 2026 · 192 findings across 8 areas

Every part of the repository was reviewed: the sequential and distributed libraries, the METIS/ParMETIS compatibility layer, esMUMPS, the command-line tools, the tests and the build/CI scripts. Instrumented builds, the full test suite under sanitizers, static analysis and fuzzing backed up the review. Severity reflects impact and how easily the bug is reached: **high** means crashes, memory corruption or silently wrong results on reachable paths.

192	36	74	82	50
Findings	High	Medium	Low + info	Confirmed by running code

Fix first

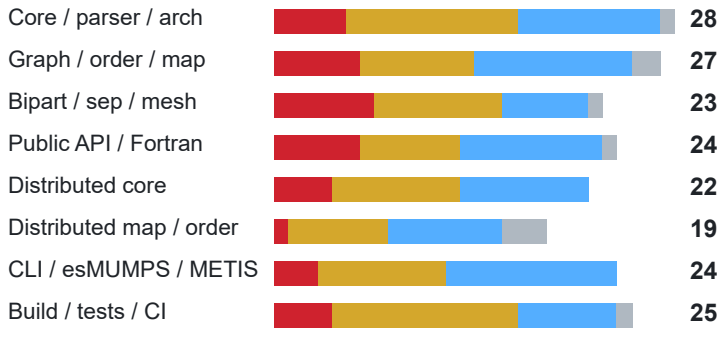
The high-severity findings with the widest reach. Severity and evidence for each are in the full list.

ID	Issue	Where	Evidence
COR-01	**Strategy parser segfaults on a syntax error inside a nested strategy parameter** Any typo in a user strategy string kills the process.	`parser_yy.y`	Reproduced by running code
GRA-04	**graphLoad2 accepts negative vertex labels and edge ends, giving out-of-bounds heap writes** Heap write at a file-controlled offset from a malformed .grf.	`graph_io.c`	Found by fuzzing
COR-02	**archLoad calls a NULL free function when a class-specific load fails** Any malformed cmplt/mesh/hcub/torus target file jumps to address 0.	`arch.c`	Reproduced by running code
GRA-05	**Remapping to more parts leaves band-graph anchor 'old part' entries uninitialised** SCOTCH_graphRemap to a larger part count crashes.	`kgraph_band.c`	Code review, re-verified against source
GRA-02	**k-way FM fallback reallocates load arrays with a stale part count** Heap overflow in the default k-way refinement when imbalance triggers the fallback.	`kgraph_map_fm.c`	Reproduced by running code
SEP-02	**meshInduceSepa reads node loads from the element-load array** Heap over-read in SCOTCH_meshOrder on element-weighted meshes.	`mesh_induce_sepa.c`	Reproduced by running code
SEP-07	**meshSave writes the wrong load arrays for elements-first meshes** Silent data corruption in saved meshes since 6.0.	`mesh_io.c`	Code review, re-verified against source
SEP-01	**Overlap partitioning dereferences NULL part array when a bipartition leaves part 1 empty** SCOTCH_graphPartOvl segfaults when a separator empties a part.	`wgraph_part_rb.c`	Reproduced by running code
SEP-05	**Diffusion bipartitioning adds the anchor injection twice and drops the reduced term** Degrades default-strategy partition quality on every run.	`bgraph_bipart_df.c`	Code review, re-verified against source
DMO-01	**hdgraphFold2 sends the wrong vertex-load slice when a sender has several receivers** Wrong vertex weights in parallel ordering on 3+ ranks.	`hdgraph_fold.c`	Code review, re-verified against source
DGR-01	**dgraphFold2 receiver failure after messages are posted skips draining requests (deadlock, use-after-free)** One rank's OOM turns into deadlock plus use-after-free.	`dgraph_fold.c`	Code review, re-verified against source

ID	Issue	Where	Evidence
DGR-04	**dgraphRedist2 corrupts the graph when a user permutation reorders vertices** Silent graph corruption through a public API call.	`dgraph_redist.c`	Code review, re-verified against source
API-02	**SCOTCH_contextOptionSetNum switches on the option value instead of the option number** 'Deterministic' mode is not deterministic.	`library_context.c`	Reproduced by running code
CLI-03	**METIS 5 ubvec (1+eps) is used unconverted as Scotch's imbalance fraction** Every METIS-API caller that sets ubvec gets ~100% imbalance.	`metis_graph_part.c`	Code review, re-verified against source
BLD-01	**CMake build silently omits five libscotch source files** CMake-built libraries miss public symbols.	`CMakeLists.txt`	Code review, re-verified against source
BLD-03	**Sonar login token is written to a file kept as a CI job artifact** CI secret exposed in downloadable artifacts; rotate it.	`analysis.sh`	Code review, re-verified against source

Findings by area and severity

Number of findings per code area (bars stacked by severity). Source: this review of v7.0.16 at b748bc4.



Area	High	Medium	Low	Info	Total
Core utilities, strategy parser & target architectures	5	12	10	1	**28**
Sequential graphs, ordering & k-way mapping	6	8	11	2	**27**
Bipartitioning, separators & meshes	7	9	6	1	**23**
Public C API & Fortran wrappers	6	7	10	1	**24**
Distributed graph & mesh core (PT-Scotch)	4	9	9	0	**22**
Distributed mapping & ordering (PT-Scotch)	1	7	8	3	**19**
CLI tools, esMUMPS & METIS/ParMETIS shim	3	9	12	0	**24**
Build system, tests & CI	4	13	7	1	**25**
All areas	**36**	**74**	**73**	**9**	**192**

Where the risk concentrates

Release builds trust their inputs. Range checks for graph, mesh, target-architecture and strategy inputs often exist only under SCOTCH_DEBUG_*, and intLoad wraps on overflow. Malformed files crash or corrupt memory instead of returning an error.

MPI error paths desynchronise ranks. A local out-of-memory or bad input on one rank often returns before a collective the others enter, or the shared error flag is misread. The job hangs or corrupts its heap instead of failing cleanly.

Recent rewrites regressed quietly. The threaded bipartitioners (diffusion, greedy growing, genetic), overlap recursive bipartitioning and the fold refactors introduced logic errors and races that the debug checkers cannot see. Several degrade default-strategy quality without any error.

API contracts drift at the edges. Fortran wrappers (by-value scalars, swapped arguments, wrong symbol case), context-bound handles that are not unwrapped, and the METIS/ParMETIS shim (ubvec meaning, -1 options, ncon) break documented behaviour.

Build and tests would not catch most of this. CMake drops five sources, several tests cannot fail, the Makefile CI step ignores make's exit status, and CI never builds 64-bit SCOTCH_Num on a 64-bit host.

How findings were verified

Method	Scope	Result
Instrumented build	CMake Debug build (SCOTCH_DEBUG_ALL) with ASan + UBSan, -Wall -Wextra, MPI and threads enabled	197 compiler warnings, triaged (77 cast-function-type, 65 missing-field-initializers, 45 maybe-uninitialized, 10 other)

Method	Scope	Result
Test suite under sanitizers	All 171 CTest tests, including MPI tests (Open MPI 4.1.2, oversubscribed)	171/171 pass functionally; UBSan flagged shift overflow, NULL pointer arithmetic and memcpy-from-NULL at 9 distinct sites
Static analysis	clang static analyzer over libscotch, libptscotch, the METIS shim and the tools	206 unique warnings, mostly MPI error-path false positives; true positives folded into findings
File-format fuzzing	Mutation fuzzing of Scotch, Chaco and MatrixMarket graphs, meshes and target architectures through gtst, gcv, mtst, mcv and atst (26 seed files)	26 unique crash signatures, reduced to the loader findings below
Strategy-string fuzzing	Mutation of mapping, ordering and partitioning strategy strings fed to the parser	10 unique failures: NULL dereference that crashes release builds, misaligned store, leaks
Targeted reproducers	Small C programs linked against debug and release libraries	Confirmed the parser and overlap-partition crashes, the sprintf stack overflow, atst min=-1, isolated-vertex separators
Parallel code review	8 review passes, one per area, over src/libscotch, libscotchmetis, esmumps, the tools, tests, CMake and CI; reviewers built their own ASan/UBSan reproducers	Every high-severity finding was reproduced or re-checked against the source; a sample of medium and low findings was re-checked too

Not covered

- The distributed loaders (dgraphLoad, dmeshLoad) were not fuzzed: the multi-rank MPI fuzz run was blocked by the sandbox policy and skipped.
- No 64-bit SCOTCH_Num (INTSIZE64) build was run, so the int/abs() truncation findings come from code review.
- Most MPI error-path findings come from code review; reproducing them needs injected allocation failures.
- MUMPS sources are not on this machine, so esmumps return-code conventions were not checked against MUMPS itself.
- Windows, macOS and FreeBSD build paths were reviewed but not built.

All findings

Grouped by area, ordered by severity, then by strength of evidence. Line numbers refer to commit b748bc4.

Core utilities, strategy parser & target architectures 28 findings · 5 high, 12 medium, 10 low, 1 info

COR-01 High **Strategy parser segfaults on a syntax error inside a nested strategy parameter**

`src/libscotch/parser_yy.y:458-487`

The mid-rule action for strategy-valued parameters sets penvptr->straptr and paraptr to NULL; if a syntax error then occurs, the PARAMVAL 'error' alternative dereferences both to build its message. SCOTCH_stratGraphMap(&s, "r{sep=m{asc=b{:width=3}}}") crashes the release library (SIGSEGV). Any typo in a user-supplied strategy string (CLI -M/-O options, solver config files) kills the process instead of returning an error.

Fix: Guard the error message against NULL straptr/paraptr (or restore them from the saved state before reporting) and YYABORT cleanly.

Evidence: Reproduced by running code · crash

COR-02 High **archLoad calls a NULL free function when a class-specific load fails**

`src/libscotch/arch.c:212-218; arch_cmplt.h:108; arch_hcub.h:104; arch_mesh.h:126, 147, 166; arch_dist.h:81`

On load failure archLoad always calls clasptr->archFree, which is NULL for cmplt, hcub, mesh2D/3D/XD, the torus variants and dist. Any malformed target file of these types jumps to address 0 instead of returning an error; 'cmplt abc' through SCOTCH_archLoad gives an ASan SEGV at pc 0x0.

Fix: Guard with if (clasptr->archFree != NULL), or call archFree(), which already does.

Evidence: Reproduced by running code · crash

COR-03 High **Sub-architecture terminal list is validated only in debug builds (heap write out of bounds)**

`src/libscotch/arch_sub.c:393-397, 444-475`

Range and duplicate checks on terminal numbers from a 'sub' target file exist only under SCOTCH_DEBUG_ARCH1; in release builds an out-of-range number indexes treetab directly, and an archDomTerm failure leaves orgdomndat uninitialised. 'sub 2 0 99 cmplt 4' gives a heap-buffer-overflow WRITE at arch_sub.c:472.

Fix: Always validate vnumnbr, ranges, duplicates and the archDomTerm result.

Evidence: Reproduced by running code · memory-safety

COR-04 High **Sub-architecture load error paths double-free and use uninitialised pointers**

`src/libscotch/arch_sub.c:115-121, 193-198, 423-427, 450-455, 482-497; arch.c:204-207`

archSubArchBuild publishes orgarchptr and termtab into subarchptr before it can fail; on failure archSubArchLoad frees orgarchptr and the outer archLoad frees it again via archSubArchFree (termtab too on some paths). If the inner archLoad fails on an unknown type, clasptr is never set and archExit reads garbage. ASan reported a double free; UBSan a 0xbebebebe class pointer.

Fix: Publish fields only on success, give each allocation one owner, and leave clasptr valid on every failure.

Evidence: Reproduced by running code · memory-safety

COR-05 High **Loading any 'dist' target file dereferences an unallocated sub-architecture pointer**

`src/libscotch/arch_dist.c:94-99`

archDistArchLoad passes archptr->archptr to archLoad, but nothing allocates it. Even the well-formed 'dist 2 cmplt 4' crashes (UBSan member access through NULL at arch.c:209, then SEGV).

Fix: Allocate and free the inner Arch, or reject 'dist' in target files.

Evidence: Reproduced by running code · crash

COR-06 Medium **Labelled tree-leaf (ltleaf) permutation is validated only in debug builds**

`src/libscotch/arch_tleaf.c:692-735, 799-801, 828-831`

The level-boundary size check and the duplicate check run only under SCOTCH_DEBUG_ARCH2. With duplicates some peritab slots stay unwritten and are later used as indices ('ltleaf 1 4 1 4 0 0 1 2' gives a heap-buffer-overflow READ at line 801); a size that does not divide the terminal count maps terminals outside the architecture. Our fuzzing also showed the debug check itself reads sizetab[-1] because the loop increment indexes before the body tests levlnum < 0.

Fix: Run both checks unconditionally and test levlnum before indexing.

Evidence: Reproduced by running code · memory-safety

COR-07 **Medium** **Zero or negative dimension/level counts accepted for meshXD and tleaf targets**

`src/libscotch/arch_mesh.c:632-636, 773-778, 843; arch_tleaf.c:115-137, 240-255`

archMeshXArchLoad only bounds dimnnbr above, and archTleafArchLoad never checks levlnbr. 'meshXD 0' makes gmap read c[-1] in archMeshXDomNum (and matching would compute % 0); 'sub 1 0 tleaf 0' reads sizetab[-1] in archTleafMatchInit. Fuzzing also showed the tleaf size product overflowing Anum (UBSan line 136) and sizetab leaking on 'bad input (2)'.

Fix: Reject counts < 1, check the size product for overflow, and free sizetab on error.

Evidence: Reproduced by running code · input-validation

COR-08 **Medium** **Fork-mode compression deadlocks with several compressed files and duplicates stdio output**

`src/libscotch/common_file_compress.c:246-262; common_file_decompress.c:224-239; common_file.c:292-301`

Without COMMON_PTHREAD, each compressed stream forks a child that inherits the write ends of earlier pipes; fileBlockClose waits for child 0 before closing pipe 1, but child 1 holds pipe 0 open, so the program hangs (reproduced with two .gz outputs). Children call exit(), flushing parent stdio buffers again (stdout written twice).

Fix: Close inherited pipe ends in the child (or use O_CLOEXEC) and call _exit().

Evidence: Reproduced by running code · deadlock

COR-09 **Medium** **threadContextInit hangs and leaks when pthread_create fails**

`src/libscotch/common_thread.c:136-147, 186-191`

After a creation failure thrdnbr is reduced and threadContextExit spin-waits for barrnbr == thrdnbr - 1, but started threads are stuck in a startup barrier sized for the original count; reproduced with an injected failure. desctab is never freed, and the mutex/condvar are not destroyed if the first creation fails.

Fix: Track threads actually started and make shutdown independent of the startup barrier.

Evidence: Reproduced by running code · deadlock

COR-10 **Medium** **A thread count of 0 (e.g. SCOTCH_PTHREAD_NUMBER=0) hangs**

`src/libscotch/common_thread.c:114-131, 149-154; common_context.c:222`

Only < 0 ('auto') and == 1 are special-cased; with 0 the code allocates zero descriptors, writes desctab[0], and waits in a barrier that never completes. The value can come straight from the environment.

Fix: Treat 0 as 1 (or reject it) wherever the thread count is read.

Evidence: Reproduced by running code · deadlock

COR-11 **Medium** **Imported thread contexts restore an affinity mask that was never saved**

`src/libscotch/common_thread.c:582-600, 203-210, 880-886; library_context.c:212-217`

SCOTCH_contextThreadImport1 never calls threadProcessStateSave, but contextExit later applies savedat.cpusdat to the caller's thread, overwriting the application thread's CPU affinity with leftover heap content (a harness was re-pinned to CPU 0).

Fix: Save state on import, or use the no-restore exit path for imported contexts.

Evidence: Reproduced by running code · threads

COR-12 **Medium** **Strategy lexer counts whitespace twice, so error messages read past the string**

`src/libscotch/parser_ll.l:90, 166-168; parser.c:637-658; parser_yy.y:283-285`

YY_USER_ACTION already calls parserLocationUpdate for every rule, and the whitespace rule calls it again, so each blank advances teenptr twice. Error messages print tebeptr with %s, which can then point past the NUL (guard-page test segfaulted; a 302-character string reported column 602).

Fix: Remove the explicit call in the whitespace rule.

Evidence: Reproduced by running code · memory-safety

COR-13 **Medium** **stratTestSave parenthesises the wrong subexpressions, so saved strategies change meaning**

`src/libscotch/parser.c:551-552, 586-595`

The parenthesis table is indexed the wrong way round: lower-precedence children get no parentheses, leaves get them. '/(vert>(10+2)*3)?m;' is saved as '(vert)>((10)+(2)*(3))', which re-parses as 16 instead of 36. Right-operand and fprintf errors are ignored.

Fix: Swap the table rows (or the condition) and propagate errors.

Evidence: Reproduced by running code · wrong-result

COR-14 **Medium** **meshXD distance uses torus wrap-around**

`src/libscotch/arch_mesh.c:911-915`

archMeshXDomDist is a copy of archTorusXDomDist and folds distances larger than the dimension; meshes have no wrap-around links. atst on 'meshXD 2 8 8' reports max distance 8 (like torus2D 8 8), while mesh2D 8 8 reports 14. Mapping onto meshXD (the only way to describe 4-5D meshes) optimises the wrong objective.

Fix: Use distval += disttmp in the mesh version.

Evidence: Reproduced by running code · wrong-result

COR-15 **Medium** **intLoad silently wraps on integer overflow**

`src/libscotch/common_integer.c:111`

val = val * 10 + digit has no overflow check, so '2147483648' or longer digit strings become negative or small values (signed-overflow UB, reported by UBSan hundreds of times while fuzzing). Every loader (graph, mesh, arch, mapping, ordering) relies on intLoad, so range checks like 'count >= 0' can be bypassed by out-of-range input. The core review reproduced it end to end: with 32-bit INT, the target 'cmplt 4294967298' loads as a 2-terminal architecture.

Fix: Detect overflow before multiplying (val > (INT_MAX - d) / 10) and return 0 (error).

Evidence: Found by fuzzing · input-validation

COR-16 **Medium** **Deco architecture loader ignores build errors and does not validate compiled decompositions**

`src/libscotch/arch_deco.c:297, 301-338, 485`

For type-0 files the return value of archDecoArchBuild2 (which can fail with 'bad terminal count', 'invalid null distance', ...) is discarded, so archDecoArchLoad reports success on a half-built architecture. For type-1 files, domain sizes, labels and the implied binary tree are never validated; a fuzzed file made archDecoDomSize read past domverttab (ASan heap-buffer-overflow).

Fix: Propagate archDecoArchBuild2's result, and validate type-1 data (sizes > 0, termdommax consistent with termdomnbr, child indices in range).

Evidence: Found by fuzzing · input-validation

COR-17 **Medium** **commGatherv/commScatterv can fail on root only, deadlocking other ranks**

`src/libscotch/comm.c:110, 147-172, 211-236`

Only the root validates count arrays and allocates; on failure (counts above INT_MAX, the case these wrappers exist for) it returns without entering the collective while others block in MPI_Gatherv/Scatterv. Scalar counts are also cast to int without range checks.

Fix: Agree on errors with an MPI_Allreduce before the collective, and range-check scalars.

Evidence: Code review · mpi

COR-18 **Low** **Per-process random seeds collide for neighbouring ranks**

`src/libscotch/common_integer.c:232, 246`

The seed is ((seed | 1) * (proc + 1)) | 1. With the fixed seed 1 used under COMMON_RANDOM_FIXED_SEED (set in all shipped Makefile.inc files), ranks 1 and 2 get identical streams, as do 3 and 4, defeating SCOTCH_randomProc.

Fix: Mix seed and rank with a proper hash (e.g. splitmix64).

Evidence: Reproduced by running code · rng

COR-19 **Low** **Hypercube dimension bound allows undefined shifts**

`src/libscotch/arch_hcub.c:104-106, 158, 242, 261, 380`

dimnnbr may equal the bit width of Anum but the code computes int 1 << dimnnbr; 'hcub 31' and 'hcub 32' trigger UBSan shift errors and give negative or garbage terminal counts.

Fix: Limit dimnnbr to bitsizeof(Anum) - 2 and shift (Anum) 1.

Evidence: Reproduced by running code · overflow

COR-20 **Low** **dummysizes dereferences NULL when given fewer than two file names**

`src/libscotch/dummysizes.c:117-120, 228-231, 406-412`

C_fileTab names are NULL unless both arguments are given, yet nameptr[0] is read unconditionally (UBSan at line 229). The SCOTCH_DEBUG_MAIN1 block uses non-existent fields, and the line-length check tests a slot fgets never fills.

Fix: Default names to "-", fix the debug block, test chartab[CHARMAX-2].

Evidence: Reproduced by running code · crash

COR-21 Low **Signed overflow in mesh/torus architecture dimension products**

`src/libscotch/arch_mesh.c:204, 479-480, 819, 891; arch_cmpltw.c:259`

Dimension products and weight sums are computed in Anum without overflow checks, so large dimensions in a .tgt file wrap (UBSan during fuzzing) and produce wrong terminal counts.

Fix: Validate dimensions at load time so that the product fits in Anum.

Evidence: Found by fuzzing · overflow

COR-22 Low **Parser stores a Strat* into a misaligned (int) parameter slot after error recovery**

`src/libscotch/parser_yy.y:477-479`

With a malformed mesh-ordering string, UBSan reports an 8-byte Strat* store to a 4-byte-aligned address: after error recovery, paraptr refers to an INT parameter while a STRATSELECT reduction runs. The parse ultimately fails, but this is a type-confused write into the method data union.

Fix: Check paraptr->typeval == STRATPARAMSTRAT before storing, and abort on mismatch.

Evidence: Found by fuzzing · ub

COR-23 Low **Strategy parser leaks partially built trees on syntax and semantic errors**

`src/libscotch/parser_yy.y (no %destructor), 180, 196, 225, 283-291, 498, 563, 698, 738, 819-828`

Bison discards semantic values on YYABORT and error recovery without a %destructor; stratParserParse frees only the top-level penvdat.straptr. LeakSanitizer flagged allocations from eight grammar actions while fuzzing invalid strategies (found by both our fuzzing and the core review).

Fix: Add %destructor declarations for the strategy, test and string value types.

Evidence: Found by fuzzing · memory-leak

COR-24 Low **Explicit core table uses a different numbering than automatic placement**

`src/libscotch/common_thread.c:140, 151, 790-793, 829-853`

coretab[i] % corenbr is used as a physical CPU id, where corenbr is the size of the process mask rather than the platform core count; with a cpuset of CPUs 8-15, coretab = {8..15} pins threads to CPUs 0-7. The setaffinity result is ignored.

Fix: Map coretab entries through the same mask-based lookup.

Evidence: Code review · threads

COR-25 Low **parserLocationUpdate tests the first character instead of the current one**

`src/libscotch/parser.c:648-649`

The loop checks *textptr instead of textptr[textidx], so newlines inside multi-character tokens count as columns and line/column numbers in error messages are wrong.

Fix: Test textptr[textidx] == '\n'.

Evidence: Code review · logic

COR-26 Low **Strategy expressions allow modulo by zero and unchecked float-to-int conversions**

`src/libscotch/parser.c:425-428; parser_ll.l:137, 141, 153`

Integer % in test expressions has no zero check, so '/(vert%0>1)?...' raises SIGFPE; literals are converted with (INT) atol / atof, silently truncating or invoking UB on overflow.

Fix: Reject zero divisors and parse literals with strtol/strtod plus range checks.

Evidence: Code review · crash

COR-27 Low **Complete-graph matching never alternates passes, and the alternate branch is broken**

`src/libscotch/arch_cmplt.c:208-237`

matcptr->passnum = passnum stores the same value, so for odd sizes the last vertex is always the unmatched one; the passnum == 1 branch would also drop vertices (5 vertices give 2 coarse vertices), so fixing only the toggle exposes a second bug.

Fix: Toggle with passnum ^ 1 and stop the pair loop at (finevertnbr + passnum) >> 1.

Evidence: Code review · logic

COR-28 Info **fiboHeapDecreaseMacro uses the caller's variable names instead of its parameters**

`src/libscotch/fibo.h:149-166`

The macro body uses treeptr/nodeptr rather than (t)/(n); it only works because its single caller uses those names.

Fix: Use the macro parameters throughout.

Evidence: Code review · latent

Sequential graphs, ordering & k-way mapping 27 findings · 6 high, 8 medium, 11 low, 2 info

GRA-01 High **Fixed vertices with zero weight make kgraphMapRbVfloMerge loop forever**

`src/libscotch/kgraph_map_rb.c:291-296, 409-412, 456-473`

The merge hash table is sized from domnnbr + vflonbr, where vflonbr counts fixed-vertex load slots; with all fixed vertices of weight 0, vflonbr = 0 but each distinct fixed part still inserts an entry, so the table fills and the probe loop never terminates. Reproduced: SCOTCH_graphMapFixed with 64 zero-weight fixed parts hangs (unit weights finish).

Fix: Size the table from the number of distinct fixed parts, independent of loads, or grow it when full.

Evidence: Reproduced by running code · hang

GRA-02 High **k-way FM fallback reallocates load arrays with a stale part count**

`src/libscotch/kgraph_map_fm.c:997, 1039-1056, 1321, 1393-1394`

kgraphMapFm caches grafptr->m.domnnbr in a const local; when imbalance triggers the fallback kgraphMapSt(r{sep=h{pass=10}}), recursive bipartitioning can create new parts (2 -> 4 in the repro), yet comploadmax/comploaddlt are reallocated with the old count (despite a comment saying it changed). ASan: heap-buffer-overflow READ in kgraphMapFmTablGet.

Fix: Re-read domnnbr/domntab after the fallback, or skip refinement when the part count changed.

Evidence: Reproduced by running code · memory-safety

GRA-03 High **orderLoad trusts the vertex count and permutation values in the file**

`src/libscotch/order_io.c:96-120`

permtab is allocated for the graph's vnodnbr, then ordeptr->vnodnbr is replaced by the count read from the file and used as the loop bound; values are never range-checked before orderPeri uses them as indices. ASan: WRITE overflows in intLoad (count larger than the graph) and in orderPeri (entry 5000).

Fix: Reject a count that differs from vnodnbr and check each entry is in range and forms a permutation.

Evidence: Reproduced by running code · memory-safety

GRA-04 High **graphLoad2 accepts negative vertex labels and edge ends, giving out-of-bounds heap writes**

`src/libscotch/graph_io.c:300-323`

With vertex labels enabled, labels are only bounded above (vlblmax). A negative label is used directly as an index in indxtab[vlbltax[v]] = vertnum, which is an out-of-bounds heap write at a file-controlled offset. Negative edge ends are read the same way at line 318. Found by fuzzing gtst with mutated .grf files (ASan heap-buffer-overflow at graph_io.c:302 and :318). Independently reproduced by the graph/ordering review (labels -5 and 3).

Fix: Reject labels and edge ends < 0 (and > vlblmax) while reading in graphLoad, before calling graphLoad2.

Evidence: Found by fuzzing · memory-safety

GRA-05 High **Remapping to more parts leaves band-graph anchor 'old part' entries uninitialised**

`src/libscotch/kgraph_band.c:267`

The band graph has domnnbr anchors (one per current part), but only the last r.m.domnnbr entries (the old mapping's count) of bandparotax are set to ~0. Remapping from 2 to 8 parts leaves garbage old-domain indices that FM dereferences: SCOTCH_graphRemap crashed with SEGV in archCmpltDomIncl from kgraphMapFmPartAdd (reproduced). With more old parts than new, the memSet overwrites ordinary band vertices.

Fix: memSet (bandparotax + bandvertnbr - domnnbr, ~0, domnnbr * sizeof (Gnum)).

Evidence: Code review, re-verified against source · crash

GRA-06 High **Conditional ordering strategy without an else branch returns success with an unset ordering**

`src/libscotch/hgraph_order_st.c:291-296; order.c:169-170`

For '/cond?s;' the else strategy is NULL; when the condition is false hgraphOrderSt writes nothing and returns 0 (the empty-strategy case calls hgraphOrderSi). Reproduced: with '/(vert>1000)?s;' on a small graph SCOTCH_graphOrder returns 0 with peritab still holding the caller's sentinel -7, and orderPeri writes permtab[-7].

Fix: Call hgraphOrderSi when stratab[1] == NULL.

Evidence: Code review, re-verified against source · memory-safety

GRA-07 **Medium** **Matrix Market loader overflows or corrupts vertex arrays when the base is not 1**

`src/libscotch/graph_io_mmkt.c:102-110, 227-229`

The gap-filling loop compares the based vertnum with the unbased mrownbr: with base 0 one extra verttax[++vertnum] write lands past the array (ASan WRITE at line 229); with base 2 graphCheck reports 'invalid vertex arrays'.

Fix: Loop to grafptr->vertnnd - 1.

Evidence: Reproduced by running code · memory-safety

GRA-08 **Medium** **Chaco loader edge-overflow check is off by one and runs too late**

`src/libscotch/graph_io_chac.c:219, 225-230`

edgenum > edgenbr + 1 lets one extra arc be written past the based array, and edge loads are stored before any check. A Chaco file with more arcs than declared overflows the heap (ASan WRITE at line 230).

Fix: Test edgenum >= edgenbr + baseval before both stores.

Evidence: Reproduced by running code · memory-safety

GRA-09 **Medium** **Scotch geometry loader never stores the dimension**

`src/libscotch/graph_io_scot.c:101-117, 197`

dimnnbr is read into a local and used to fill geomtab but never assigned to geomptr->dimnnbr: after SCOTCH_graphGeomLoadScot, SCOTCH_geomData reports dimension 0 and SCOTCH_graphGeomSaveScot writes no coordinates (reproduced). An existing geomtab is reused without checking its size.

Fix: Set geomptr->dimnnbr after validation and reallocate geomtab when the dimension changes.

Evidence: Reproduced by running code · wrong-result

GRA-10 **Medium** **graphLoad never range-checks edge ends of unlabelled graphs in release builds**

`src/libscotch/graph_io.c:244-249, 271-277`

Edge end values are stored as read (plus baseadj). The only validation is graphCheck, which is compiled in only under SCOTCH_DEBUG_GRAPH2. A library user calling SCOTCH_graphLoad and then mapping or ordering a malformed file indexes arrays out of bounds.

Fix: Check baseval <= end < vertnnd inside the edge loop; it costs one comparison per arc.

Evidence: Code review · input-validation

GRA-11 **Medium** **Harwell-Boeing graph loader does not validate row indices**

`src/libscotch/graph_io_habo.c:266-280`

habnzrval is decoded from fixed-width columns and used directly as an index into verttax; a blank field (0), a row beyond the order, or non-digits give an out-of-range increment.

Fix: Reject values outside [1, habcolnbr] and non-digit characters.

Evidence: Code review · memory-safety

GRA-12 **Medium** **Diffusion mapping threads can wait on different barriers**

`src/libscotch/kgraph_map_df.c:222-231, 256-258, 287-289, 528-530`

A thread that finds a zero-degree anchor or fails to allocate sorttab jumps to fail: inside the pass loop, skipping the two setup barriers; the others pass the first and wait forever at the second. This loop is on the default mapping strategy (d{pass=40}); bgraph_bipart_df.c handles the same case correctly.

Fix: Route setup failures through both barriers (as bgraph_bipart_df.c's skip: label does) and test abrtval afterwards.

Evidence: Code review · deadlock

GRA-13 **Medium** **'mdeg' ordering-strategy variable is read as a double but is a Gnum**

`src/libscotch/hgraph_order_st.c:231-234; parser.c:437`

mdeg is declared STRATPARAMDOUBLE but points to s.degrmax (a Gnum); the parser reads it through *(double *), so any ordering condition on mdeg compares garbage (and reads past the field with 32-bit Gnum).

Fix: Declare mdeg as STRATPARAMINT, like vert and edge.

Evidence: Code review · ub

GRA-14 **Medium** **graphCoarsen2 reports success after out-of-memory during coarsening**

`src/libscotch/graph_coarsen.c:254-258, 368-371, 552-556`

When edgetax allocation fails, verttax is freed but not reset; when a per-thread hash allocation fails, coarse arrays are never filled. graphCoarsen2 only checks verttax == NULL and returns 0, so callers use freed or uninitialised coarse graphs and graphExit double-frees.

Fix: NULL verttax after freeing and return retuval (after graphExit) whenever it is non-zero.

Evidence: Code review · memory-safety

GRA-15 **Low** **k-way FM fallback strategy is never freed and its result is unchecked**

`src/libscotch/kgraph_map_fm.c:1039-1040`

Each fallback parses a new strategy with stratInit that is never freed (LeakSanitizer confirmed), NULL is not checked, and kgraphMapSt's return value is ignored.

Fix: Build the strategy once (or stratExit it), check NULL, propagate the result.

Evidence: Reproduced by running code · memory-leak

GRA-16 **Low** **Signed 1 << 31 in hash and flag bitmaps (undefined behaviour)**

`hgraph_order_cp.c:186; hmesh_order_cp.c:180; kgraph_map_bd.c:304, 311; library_dgraph_map_stat.c:432; mesh_induce_sepa.c:161`

Bit sets are int arrays updated with (1 << bit) for bit in 0..31; bit 31 overflows int. UBSan reported this 40+ times across the test suite (gord, gmap, mord, dgmap tests). Compilers may legally miscompile it.

Fix: Use unsigned arrays and (1U << bit).

Evidence: Sanitizer report during test suite · ub

GRA-17 **Low** **kgraphMapFmEdgeResize publishes the new size before reallocating; caller ignores failure**

`src/libscotch/kgraph_map_fm.c:524-529, 712`

The doubled size is written back before memRealloc succeeds, and the caller ignores the return value and stores into the next slot, past the old buffer on failure.

Fix: Update the size only after success and propagate the error.

Evidence: Code review · memory-safety

GRA-18 **Low** **Diffusion mapping keeps the previous pass's result**

`src/libscotch/kgraph_map_df.c:525-527, 537-539`

Each pass writes difntax and then swaps, so the latest values are in difotax, but the final copy reads difntax (one pass never changes the mapping). bgraph_bipart_df.c reads difotax correctly.

Fix: Copy from difotax[vertnum].partval.

Evidence: Code review · wrong-result

GRA-19 **Low** **kgraphCost isolated-anchor handling never runs**

`src/libscotch/kgraph_cost.c:150-154`

The search loop uses continue instead of break, so it always ends with domnnum == domnnbr and the isolated-anchor block is dead; balance for those domains is computed as if they had no load.

Fix: Use '== 0) break;' as kgraph_map_bd.c does.

Evidence: Code review · logic

GRA-20 **Low** **kgraphMapBd part-count check can never fail, and an OOM path leaks**

`src/libscotch/kgraph_map_bd.c:180-196, 268-271`

orggrafptr->m.domnnbr is overwritten with the band graph's value just before the two are compared, so a band strategy that adds parts is not detected; the orgflagtab allocation failure leaks bndvnumtax and bndgrafdat.

Fix: Compare against the saved count before copying back; free on the failure path.

Evidence: Code review · logic

GRA-21 **Low** **kgraphMapRbBgraph leaks the bipartition graph on error**

`src/libscotch/kgraph_map_rb.c:547-550, 667-670`

After bgraphInit succeeds, the veextax allocation failure and the fail: path return without bgraphExit, and the callers do not free it either.

Fix: Call bgraphExit on these error returns.

Evidence: Code review · memory-leak

GRA-22 Low **hgraphOrderBl sums weights in vertex-index order, not ordering order**

`src/libscotch/hgraph_order_bl.c:132, 161-164`

velotax is indexed by local vertex number but vnodnum walks ordering positions, so sub-block boundaries balance the wrong vertices and produce unbalanced blocks on weighted graphs.

Fix: Map each ordering position back to its vertex through peritab before reading velotax.

Evidence: Code review · wrong-result

GRA-23 Low **archDomTerm failures ignored when building domains from part arrays**

`src/libscotch/mapping.c:389; kgraph_map_rb.c:466`

mapBuild3 (old/remap parts) and kgraphMapRbVfloMerge (fixed parts) do not check archDomTerm; an invalid terminal number leaves an uninitialised ArchDom that later code uses (the same call is checked elsewhere).

Fix: Check the return value and fail with 'invalid part array'.

Evidence: Code review · error-handling

GRA-24 Low **Harwell-Boeing format parser has an isdigit typo**

`src/libscotch/graph_io_habo.c:407`

isdigit (*charptr < '0') tests a comparison result, so formats with a skip prefix such as (1X,10I8) are always rejected.

Fix: isdigit ((unsigned char) *charptr).

Evidence: Code review · logic

GRA-25 Low **hgraphOrderCp leaks the coarse ordering tree when compressed ordering fails**

`src/libscotch/hgraph_order_cp.c:449-454`

On failure only coarperitab is freed; the column-block tree already attached to coarordedat is not released (the signed 1 << 31 at line 186 in the same file is covered by the shift-overflow finding).

Fix: Call orderExit (&coarordedat) on the error path.

Evidence: Code review · memory-leak

GRA-26 Info **Base-adjusting NULL pointers ('ptr - baseval') is formally undefined**

`kgraph_map_ml.c:239; bgraph_bipart_ml.c:195 (pervasive idiom)`

Computing coarmulttab - baseval when coarmulttab is NULL is pointer arithmetic on NULL (UBSan, 30+ reports). It works on all current compilers, but it also defeats NULL checks done after offsetting (see the gout Tulip finding).

Fix: Offset only after a NULL check, or keep a separate flag.

Evidence: Sanitizer report during test suite · ub

GRA-27 Info **graphDump emits the wrong array name for edge loads**

`src/libscotch/graph_dump.c:131`

The generated C code refers to 'delotab' while the array is emitted as 'edlotab', so dumped code for graphs with edge loads does not compile.

Fix: Change delotab to edlotab.

Evidence: Code review · typo

Bipartitioning, separators & meshes

23 findings · 7 high, 9 medium, 6 low, 1 info

SEP-01 High **Overlap partitioning dereferences NULL part array when a bipartition leaves part 1 empty**

`src/libscotch/wgraph_part_rb.c:355-363, 297-303`

When the vertex separator leaves part 1 empty, wgraphPartRb2 recurses with spltnum = partval = 0 (despite the 'use fake part 1' comment), parttax = NULL and an uninitialised splttab[0].vertnbr. If that half has one domain, wgraphPartRb3One reads orgparttax[] through NULL. Reproduced via SCOTCH_graphPartOvl on a 6-vertex path, 3 parts, strategy "r{sep=z}": release build segfaults; debug build logs 'invalid parameters', leaves parttax unfilled and then hits a heap-buffer-overflow read in wgraphCost (wgraph.c:215). For 4+ parts it logs internal errors but still returns 0. Independently reproduced by the bipartition/separator review, which traced it to commit 1a52566.

Fix: Call wgraphPartRb2 (contptr, 1, &spltdat) on the fake part as the comment intends, and make the Rb3* helpers report errors through revaptr instead of returning void.

Evidence: Reproduced by running code · crash

SEP-02 **High** **meshInduceSepa reads node loads from the element-load array**

`src/libscotch/mesh_induce_sepa.c:180-186, 227-233, 347; reached from hmesh_order_nd.c:213`

For meshes with element weights, the separator mesh reads orgmeshptr->velotax[orgvnodnum] (element loads indexed by a node number) and stores it in the induced mesh's velotax, over-reading at every nested-dissection level; real node weights are ignored and vnlosum is inconsistent. Reproduced: ASan heap-buffer-overflow from SCOTCH_meshOrder on an 8x8 element-weighted quad mesh.

Fix: Read orgmeshptr->vnlotax, write indmeshptr->vnlotax, leave velotax NULL.

Evidence: Reproduced by running code · memory-safety

SEP-03 **High** **Mesh multilevel separation never stops when coarsening stalls (stack overflow)**

`src/libscotch/vmesh_separate_ml.c:93-98; mesh_coarsen.c:99-100, 350, 397-399`

meshCoarsen ignores its coarnbr/coarrat arguments and always returns 0, so recursion stops only when vnodnbr <= vnod. A mesh that cannot shrink (e.g. many disconnected elements) is rebuilt identically each level: the default SCOTCH_meshOrder on 300 disjoint triangles hits an ASan stack-overflow. If the matching hash allocation fails, finemulttax is used uninitialised.

Fix: Return 1 when the coarse mesh exceeds coarrat of the fine mesh (as graphCoarsen does) and report matcher allocation failures.

Evidence: Reproduced by running code · crash

SEP-04 **High** **Genetic bipartitioner divides by zero on 2-vertex graphs**

`src/libscotch/bgraph_bipart_ga.c:239 (guard at 293-294)`

The guard only rejects vertnbr < 2; with exactly 2 vertices contextIntRandVal(contptr, 0) computes % 0 (SIGFPE reproduced). Recursive bipartitioning regularly produces 2-vertex subgraphs, so any strategy using 'a' can crash.

Fix: Return early when vertnbr < 3, or use crossover point 1.

Evidence: Reproduced by running code · crash

SEP-05 **High** **Diffusion bipartitioning adds the anchor injection twice and drops the reduced term**

`src/libscotch/bgraph_bipart_df.c:380 (values reduced at 367-372 never read)`

vancval = loopptr->vanctab[i] + loopptr->vanctab[i] adds the constant injection to itself instead of adding the per-thread reduced external-gain contribution, so every diffusion run injects twice the intended amount and loses the veextax coupling used in mapping. Introduced by commit 67c0d61 (2020); diffusion is part of the default band strategies, so this degrades partition quality rather than crashing.

Fix: vancval = loopptr->vanctab[i] + loopptr->thrdtab[thrdnum].vanctab[i].

Evidence: Code review, re-verified against source · wrong-result

SEP-06 **High** **Threaded greedy graph growing keeps the last improving pass, not the best one**

`src/libscotch/bgraph_bipart_gg.c:269-276`

Each thread saves a pass when it beats grafptr->commload/compload0dlt, but those fields still hold the input partition (written only after the reduction, around line 432). Any later pass that beats the input overwrites a better earlier pass. Regression from commit 7a934a8.

Fix: For passnum > 0, compare against thrdptr->cmloval and ABS(thrdptr->cpl0dlt).

Evidence: Code review, re-verified against source · wrong-result

SEP-07 **High** **meshSave writes the wrong load arrays for elements-first meshes**

`src/libscotch/mesh_io.c:351-357`

velotabtab[0] is assigned twice and velotabtab[1] never: elements are written with node loads, and nodes through an uninitialised stack pointer. Reproduced: element loads 1/2/3 were saved as all 1. Also affects meshGeomSaveScot; present since 6.0.

Fix: Line 356 should be velotabtab[1] = meshptr->vnlotax.

Evidence: Code review, re-verified against source · wrong-result

SEP-08 **Medium** **Harwell-Boeing mesh loader trusts file indices in release builds and never sets velosum**

`src/libscotch/mesh_io_habo.c:180-186, 204-207, 235-242, 282`

ELTPTR/VARIND range checks exist only under SCOTCH_DEBUG_MESH2, so malformed files drive verttax[habnodval]++ and edgetax writes out of bounds. velosum stays 0, which meshCheck rejects (reproduced).

Fix: Make the checks unconditional and set velosum = habeltnbr.

Evidence: Reproduced by running code · input-validation

SEP-09 **Medium** **bgraphBipartFm keeps using its hash table after freeing it**

`src/libscotch/bgraph_bipart_fm.c:598-604`

When resizing fails, hashtab and the gain table are freed but the code neither returns nor jumps; line 604 probes the freed table and final cleanup frees it again.

Fix: return (1) after the cleanup.

Evidence: Code review, re-verified against source · memory-safety

SEP-10 **Medium** **bgraphZero counts fixed load in compload0, contradicting bgraphCheck**

`src/libscotch/bgraph.c:246-249; bgraph_check.c:182`

Everywhere else compload0 excludes fixed vertices; bgraphZero adds vfixload[0], so with fixed vertices compload0 and compload0dlt are too high and refinement starts from the wrong imbalance.

Fix: compload0 = grafptr->s.velosum.

Evidence: Code review · logic

SEP-11 **Medium** **Greedy-growing thread reduction compares signed imbalance**

`src/libscotch/bgraph_bipart_gg.c:375-377`

On a communication-load tie the reduction prefers the smaller signed cpl0dlt, so a heavily underloaded part 0 beats a balanced one; the per-thread test uses absolute values.

Fix: Compare ABS() of both deltas.

Evidence: Code review · logic

SEP-12 **Medium** **Genetic bipartitioner threads race on the sort table and a shared RNG**

`src/libscotch/bgraph_bipart_ga.c:177, 190-205, 235-249`

No barrier separates the offspring loop from the next generation's cost loop, so a fast thread can overwrite sorttab[k].parttab while another reads it as a parent. All threads call contextIntRandVal on one shared non-atomic state, so results are not reproducible.

Fix: Add threadBarrier after the offspring loop and give each thread its own generator (intRandSpawn).

Evidence: Code review · threads

SEP-13 **Medium** **Greedy-growing workers spawn their RNG while thread 0 is using it**

`src/libscotch/bgraph_bipart_gg.c:175-176, 196`

Workers call intRandSpawn on the shared contptr->randptr while thread 0 advances it, so seeds depend on timing (data race, non-reproducible runs even in 'deterministic' mode).

Fix: Spawn all per-thread generators before launching threads.

Evidence: Code review · threads

SEP-14 **Medium** **Diffusion NaN and abort paths break thread synchronisation and are ignored**

`src/libscotch/bgraph_bipart_df.c:207-213, 356-366, 519-526`

On NaN, vertnum = vancnnt is followed by a store into the next thread's first vertex and a read past the thread-local ielstax; the thread then skips threadReduce while others wait (hang when veextax != NULL). abrtval is never checked, so the routine reads uncomputed results and returns 0.

Fix: Leave the loop before the store, still reach the reduction, and return 1 when abrtval != 0.

Evidence: Code review · threads

SEP-15 **Medium** **hmeshOrderNd frees an interior pointer on the error path**

`src/libscotch/hmesh_order_nd.c:155-157, 213-216`

frontab is the second member of an allocation group led by parttax, so memFree(frontab) passes an interior pointer to free(); nspmeshdat.m.vendtax also leaks.

Fix: Call vmeshExit(&nspmeshdat) instead.

Evidence: Code review · memory-safety

SEP-16 **Medium** **meshGeomLoadScot reads node labels from the wrong range**

`src/libscotch/mesh_io_scot.c:169`

Copied from graph_io_scot.c: node labels live in [vnodbas, vnodnnd) but the loader indexes from baseval; for elements-first meshes coordinates are matched against element labels ('geometry not found' or misplaced coordinates).

Fix: Index with vnodnum + meshptr->vnodbas.

Evidence: Code review · off-by-one

SEP-17 Low **Overlap partitioning puts isolated vertices in the separator, assigned to no part**

`src/libscotch/wgraph_part_rb.c (default PartOvl strategy)`

SCOTCH_graphPartOvl on graphs with isolated vertices returns -1 (separator) for them; e.g. 2 isolated vertices with 3 parts gives {2, -1}, and 3 vertices with 6 parts gives {-1, -1, -1}. A separator vertex belongs to its neighbours' parts, so an isolated one belongs to none and is effectively unassigned.

Fix: Force zero-degree vertices out of the separator (assign them to the lighter side) in the vertex-separation strategies.

Evidence: Reproduced by running code · logic

SEP-18 Low **Leftover debug output in library code**

`src/libscotch/mesh_coarsen.c:431; wgraph_part_fm.c:857-858`

Element-weighted mesh coarsening prints '++ ...' to stderr for every overweight element (marked TODO REMOVE), and wgraphPartFm prints 'OUCH' just before an unguarded out-of-bounds savetab write.

Fix: Remove the prints and turn the savenbr test into a real check.

Evidence: Reproduced by running code · other

SEP-19 Low **bgraphBipartGg debug consistency check result is discarded**

`src/libscotch/bgraph_bipart_gg.c:437-444`

Under SCOTCH_DEBUG_BGRAPH2, a failing bgraphCheck sets o = 1, which the next line overwrites with o = 0. The check that should catch races in the recently multi-threaded greedy-graph-growing code can therefore never fail. Flagged by clang's dead-store checker. Also reported by the bipartition/separator review.

Fix: Move o = 0 before the debug block (or return directly on check failure).

Evidence: Static analyzer + code review · error-handling

SEP-20 Low **64-bit Gnum load deltas truncated by abs() and int in sequential bipartitioners**

`bgraph_bipart_gg.c:221, 272; bgraph_bipart_ga.c:204, 259; bgraph_bipart_fm.c:136, 210, 668, 677; vgraph_separate_fm.c:132, 196, 336, 594-595; vmesh_separate_fm.c:267, 906; wgraph_part_rb.c:293`

About 30 sites pass Gnum deltas to libc abs(int), and wgraphPartRb2 declares indvertnbr as int; with 64-bit Gnum, values >= 2^31 are truncated, breaking balance comparisons.

Fix: Use ABS() from common.h and Gnum for indvertnbr.

Evidence: Code review · overflow

SEP-21 Low **hmeshMesh memCpy size has an operator-precedence error**

`src/libscotch/hmesh_mesh.c:130`

The size evaluates as vnodnbr - vnohnbr * sizeof(Gnum), usually negative, i.e. a huge size_t. Only the nodes-first halo branch runs it, which current callers do not use (latent).

Fix: (vnodnbr - vnohnbr) * sizeof(Gnum).

Evidence: Code review · ub

SEP-22 Low **Error-path leaks and an ignored return value in separators**

`vgraph_separate_fm.c:202, 587-590; mesh_induce_sepa.c:205-207; hmesh_order_nd.c:157-159`

vgraphSeparateFm returns on resize failure without freeing its tables and ignores vgraphSeparateGg's result; meshInduceSepa and hmeshOrderNd leak on allocation failure.

Fix: Route through the existing cleanup and propagate the result.

Evidence: Code review · memory-leak

SEP-23 Info **meshBase and hmeshBase do not rebase consistently**

`src/libscotch/mesh.c:175-182; hmesh.c:106-111`

meshBase does not shift velotax/vnlotax, and hmeshBase loops over the new element range while vehdtax is still based on the old one. No callers today.

Fix: Shift all arrays and rebase vehdtax before looping.

Evidence: Code review · latent

Public C API & Fortran wrappers 24 findings · 6 high, 7 medium, 10 low, 1 info

API-01 High **STRATGRAPHORDERBUILD Fortran wrapper swaps flagval and levlnbr**

`src/libscotch/library_graph_order_f.c:331-341`

The wrapper passes *levlnbr and *flagval to SCOTCH_stratGraphOrderBuild in the wrong order. With flagval = SCOTCH_STRATLEVELMAX and levlnbr = 7, the C call yields 'sep=(/(((levl)<(7))&((vert)>(240)))?...' while the Fortran entry yields 'sep=(/((vert)>(240))?...': the level limit silently disappears.

Fix: Call SCOTCH_stratGraphOrderBuild (straptr, *flagval, *levlnbr, *kbalval).

Evidence: Reproduced by running code · fortran

API-02 High **SCOTCH_contextOptionSetNum switches on the option value instead of the option number**

`src/libscotch/library_context.c:306-323, 368`

switch (optival) should be switch (optinum). Setting SCOTCH_OPTIONNUMDETERMINISTIC = 1 (including via 'd' in SCOTCH_contextOptionParse) never forces a fixed random seed, so 'deterministic' runs are not deterministic; any value other than 0/1 is rejected as 'invalid option name'. The Parse caller also ignores the return value.

Fix: switch (optinum), and propagate the result in SCOTCH_contextOptionParse.

Evidence: Reproduced by running code · logic

API-03 High **SCOTCH_graphMapView overwrites the caller's mapping and computes distances on renumbered parts**

`src/libscotch/library_graph_map_view.c:290, 303, 316, 465-466`

parttax points at the user's parttab and is rewritten in place with renumbered parts 0..mapnbr-1, never restored; archDomTerm is then called on the renumbered indices. An 8-vertex path mapped to {0,0,0,0,7,7,7,7} on mesh2D 8x1 reported dilation 1 (true: 7), and parttab came back as {0,0,0,0,1,1,1,1}. Affects MapView, RemapView and the Raw variants.

Fix: Renumber into private memory and keep the original parts for distance calculations.

Evidence: Reproduced by running code · wrong-result

API-04 High **SCOTCH_dgraphGather lets non-root processes write into their own Dgraph**

`src/libscotch/library_dgraph_gather.c:86-88, 121-123`

In Fortran a non-root rank passes its distributed graph as cgrfptr to mean 'not root'. The routine recognises this when counting roots, but then passes the raw pointer to dgraphGatherAll2, which treats any non-NULL pointer as root and fills Graph fields inside the Dgraph structure, corrupting it in release builds. A 2-rank debug run failed with 'should have only one root'.

Fix: Normalise cgrfptr once (aliases to NULL, otherwise CONTEXTOBJECT) and pass that value.

Evidence: Reproduced by running code · memory-safety

API-05 High **Three Fortran wrappers take scalar arguments by value**

`src/libscotch/library_graph_induce_f.c:71, 77, 87-89, 94; library_dgraph_band_f.c:72-74, 79`

Fortran passes every argument by address, and the manuals document vnumnbr, partval, seedlocnbr and distval as integer*num. GRAPHINDUCELIST, GRAPHINDUCEPART and DGRAPHBAND declare them as values, so the C routine receives an address as the vertex count or band distance (or the low byte of an address as partval): huge counts, reads past vnumtab/fronloctab, crashes or garbage graphs.

Fix: Declare the parameters as pointers and dereference them, as every other wrapper does.

Evidence: Code review, re-verified against source · fortran

API-06 High **Mesh routines read context containers as Mesh structures (heap overflow in mesh ordering)**

`src/libscotch/library_mesh_order.c:110-118, 147, 164, 181; library_mesh.c:157, 172, 279, 338, 531, 567, 628; library_mesh_graph.c:82, 99`

SCOTCH_contextBindMesh creates a container, but SCOTCH_meshOrderInit and the other mesh entry points cast it straight to Mesh* (only SCOTCH_meshOrderComputeList unwraps it). For a bound mesh vnodnbr reads as 0, orderInit allocates an empty peritab and hmeshOrderSt writes the real node count into it.

Fix: Use CONTEXTOBJECT (meshptr) in every mesh entry point.

Evidence: Code review, re-verified against source · memory-safety

API-07 **Medium** **Strategy builders sprintf a double into 32/64-byte stack buffers**

`library_graph_map.c:669-670, 730-731; library_graph_part_ovl.c:169; library_graph_order.c:514; library_mesh_order.c:414; library_dgraph_map.c:307-308, 381-382; library_dgraph_order.c:403`

sprintf(buf, "%lf", ratio) writes about 300 characters for large values. Confirmed with ASan: SCOTCH_stratGraphMapBuild, SCOTCH_stratGraphOrderBuild and SCOTCH_stratGraphPartOvlBuild with balrat = 1e300 each report a stack-buffer-overflow (WRITE of size 309). Callers passing a 'no constraint' sentinel such as DBL_MAX smash the stack. Also found independently by the public-API review.

Fix: Use snprintf(buf, sizeof(buf), "%g", v) and reject non-finite or out-of-range ratios.

Evidence: Reproduced by running code · memory-safety

API-08 **Medium** **MESHGEOMLOADHABO passes NULL where the loader requires a string**

`src/libscotch/library_mesh_io_habo_f.c:115; mesh_io_habo.c:94`

meshGeomLoadHabo immediately calls atol(dataptr), but the Fortran wrapper always passes NULL; UBSan reported a null argument and the process aborted, so the routine cannot work from Fortran.

Fix: Pass "0", or treat NULL as matrix 0.

Evidence: Reproduced by running code · fortran

API-09 **Medium** **DGRAPHSTAT Fortran wrapper exports a symbol with the wrong case**

`src/libscotch/library_dgraph_stat_f.c:70-71`

The lower-case macro argument is 'Dgraphstat', so nm shows scotchfDgraphstat, scotchfDgraphstat_ and scotchfDgraphstat__; gfortran/ifort callers of scotchfdgraphstat fail to link.

Fix: Change the argument to dgraphstat.

Evidence: Reproduced by running code · fortran

API-10 **Medium** **Architecture builders skip the checks the file loaders perform**

`src/libscotch/library_arch.c:339-358 (Hcub), 510-525 (Tleaf), 552-562 (Ltleaf)`

SCOTCH_archLtleaf uses permtab[i] as an index into peritab without checking [0, permnbr) (heap write OOB, while archLtleafArchLoad checks it). SCOTCH_archHcub accepts any dimnnbr (1 << dim overflow). SCOTCH_archTleaf accepts levlnbr <= 0 and non-positive sizes.

Fix: Apply the loaders' range checks and free arrays on failure.

Evidence: Code review · memory-safety

API-11 **Medium** **DGRAPHMAPSTAT Fortran wrapper has no ierr argument and discards errors**

`src/libscotch/library_dgraph_map_stat_f.c:66-95`

The manual documents a trailing ierr, but the wrapper ends at cmexsumptr and throws away SCOTCH_dgraphMapStat's result; the caller's ierr is never written.

Fix: Add an int * revaptr parameter and store the result.

Evidence: Code review · fortran

API-12 **Medium** **Other entry points ignore context-bound input and output handles**

`library_dmesh_dgraph.c:78; library_graph_dump.c:87; library_mesh_graph.c:82, 99; library_dgraph_gather.c:121-123`

These cast handles straight to Dgraph*/Graph* instead of CONTEXTOBJECT; if the output graph was bound to a context, dmeshDgraphDual and meshGraph* write a whole graph over the small container, and SCOTCH_graphDump prints container bytes as graph fields.

Fix: Use CONTEXTOBJECT on these handles.

Evidence: Code review · logic

API-13 **Medium** **ARCHSIZE writes a SCOTCH_Num through an int ***

`src/libscotch/library_arch_f.c:217-224`

The manual declares the output as integer*num, but the wrapper stores through int*; with 64-bit integers the upper half is uninitialised and large sizes are truncated.

Fix: Declare sizeptr as SCOTCH_Num * const.

Evidence: Code review · fortran

API-14 Low **Part diameter in map views is off by one**

`src/libscotch/library_graph_map_view.c:129-136`

rootnum (base-adjusted) is compared with vertnbr, so with base 1 the last vertex never starts a component; distances start at 1 (4-vertex paths reported PartDiam=4, true 3); a -1 OOM return is added to the average unchecked.

Fix: Compare with vertnnd, start at 0, handle -1.

Evidence: Reproduced by running code · off-by-one

API-15 Low **SCOTCH_dgraphCoarsen calls memcpy with a NULL source**

`src/libscotch/library_dgraph_coarsen.c:163`

When folding produces an empty local multinode array, memCpy(multloctab, NULL, 0) is executed. Passing NULL to memcpy is undefined even with size 0 (UBSan in test_scotch_dgraph_coarsen).

Fix: Skip the copy when vertlocnbr == 0.

Evidence: Sanitizer report during test suite · ub

API-16 Low **Remap view's migration distance is always zero**

`src/libscotch/library_graph_map_view.c:487-488`

Both terminal domains are built from the old mapping (parotax twice), so archDomDist always returns 0 and the average migration distance is meaningless.

Fix: Build domnorg from parttax[vertnum] using the original part numbers.

Evidence: Code review, re-verified against source · wrong-result

API-17 Low **SCOTCH_dgraphBand out-of-memory path dereferences NULL and overflows a scratch buffer**

`src/libscotch/library_dgraph_band.c:142, 200-203, 247`

Same pattern as the internal dgraphBand (separate copy of the code): procdsptab is written when NULL, procglbnbr GNUM_MPI values are gathered into a buffer sized with sizeof(int), and the halo-exchange failure path leaks every allocation.

Fix: Use a local Gnum error flag, size the buffer with sizeof(Gnum), free on all paths.

Evidence: Code review · memory-safety

API-18 Low **Distributed map view's CommDelta formula is wrong**

`src/libscotch/library_dgraph_map_view.c:145-148`

The sequential view computes (expansion x edgenbr) / (load x dilation); the distributed view prints edgeglbnbr / (2 x load), ignoring expansion and dilation.

Fix: Use (cmexsum * edgeglbnbr) / (cmlosum * cmdisum).

Evidence: Code review · wrong-result

API-19 Low **graphStat and dgraphStat report the edge-load sum at two different scales**

`src/libscotch/library_graph.c:495, 510; library_dgraph_stat.c:265`

With weights, edlosum adds every arc (each edge twice); without weights it is edgenbr / 2 (each edge once), so unit weights report twice the unweighted value.

Fix: Use one convention in both branches.

Evidence: Code review · wrong-result

API-20 Low **SCOTCH_dgraphGrow leaks its temporary context and mislabels errors**

`src/libscotch/library_dgraph_grow.c:116-127`

When dgraphGhst fails, it returns without CONTEXTEXIT (leaking the context and thread pool); its context error message says 'SCOTCH_dgraphBand'.

Fix: goto fail instead of return, and fix the message.

Evidence: Code review · memory-leak

API-21 Low **SCOTCH_graphTabLoad reads one element past mappsorttab**

`src/libscotch/library_graph_map_io.c:163-171`

After the last mapping entry matches, the next while reads mappsorttab[mappfilenbr] before the bound check; with duplicate labels this can assign a wrong part.

Fix: Test the bound before reading.

Evidence: Code review · memory-safety

API-22 Low **SCOTCH_graphPartOvlView continues after an allocation failure**

`src/libscotch/library_graph_map_view.c:709-715`

The error is printed but the routine does not return, then increments and fills the NULL listtab.

Fix: return (1) in the error branch.

Evidence: Code review · error-handling

API-23 Low **Ignored *Init results and inconsistent error codes**

`library_graph_map.c:411, 438, 468-469, 501-502; library_graph_order.c:447; library_mesh_order.c:319, 351; library_arch_build.c:131; library_graph_coarsen.c:98-100`

SCOTCH_graphMap variants, SCOTCH_graphOrderList and SCOTCH_meshOrder/List ignore their own *Init result (continuing on NULL arrays); SCOTCH_archBuild0 ignores stratInit; SCOTCH_graphCoarsen returns 1 ('not coarsened enough') on context failure instead of 2.

Fix: Check every Init result and return documented codes.

Evidence: Code review · error-handling

API-24 Info **Fortran routines documented in the manuals have no wrapper**

`doc/src/scotch/s_l.tex; doc/src/ptscotch/p_l.tex`

scotchfarchbuild0, archbuild2, archltleaf, graphorderload, meshstat, stratsizeof, mapsizeof, ordersizeof, dgraphfree, dgraphgrow, dmeshfree, dmapsizeof and dordersizeof are documented but defined in no *_f.c file.

Fix: Add the wrappers or remove them from the manuals.

Evidence: Code review · fortran

Distributed graph & mesh core (PT-Scotch) 22 findings · 4 high, 9 medium, 9 low, 0 info

DGR-01 High **dgraphFold2 receiver failure after messages are posted skips draining requests (deadlock, use-after-free)**

`src/libscotch/dgraph_fold.c:463-465, 762-810`

The refactor moved the receiver's edge-array allocation to line 463, after senders posted their Isends and receivers posted vertex Irecvs. On failure, receivers jump fail3 -> fail4, skipping MPI_Waitall and the frees: senders wait on unmatched Isends while receivers sit in MPI_Allreduce (deadlock for large messages), dgraphExit frees vertloctax with Irecvs still targeting it, and several work arrays leak.

Fix: Agree on allocation success before posting edge-data Isend/Irecv, or have the fail3 path complete every posted request before fail4.

Evidence: Code review, re-verified against source · mpi

DGR-02 High **dgraphFold2 error paths free folded-graph arrays with the wrong base**

`src/libscotch/dgraph_fold.c:194, 239, 250-251, 304, 751, 809`

fldgrafptr is zeroed at line 194 (baseval = 0) but its arrays are offset by orggrafptr->baseval; the real baseval is stored only at line 751. Any dgraphExit(fldgrafptr) before that frees ptr + 0 on an offset pointer, corrupting the heap on every receiver when baseval = 1.

Fix: Set fldgrafptr->baseval = orggrafptr->baseval right after the memSet.

Evidence: Code review, re-verified against source · memory-safety

DGR-03 High **dgraphLoadCent ignores header errors, and a bad property value overflows a stack buffer**

`src/libscotch/dgraph_io_load.c:218-235, 249, 261`

On a bad header (short read or property value outside 0..111) the root sets cheklocval but broadcasts reduglbtab[0], which stays 0, so all ranks continue with garbage header values. sprintf(proptab, "%3.3d", v) then writes up to 12 bytes into char proptab[4]. dgraphLoadDist handles this correctly.

Fix: Set reduglbtab[0] = 1 on header errors and use snprintf / validate before formatting.

Evidence: Code review, re-verified against source · memory-safety

DGR-04 **High** **dgraphRedist2 corrupts the graph when a user permutation reorders vertices**

`src/libscotch/dgraph_redist.c:350-357, 425-469; library_dgraph_redist.c:100-102`

Destination edge array, send buffer and receive buffer share one block; the 'never overlap' TRICK holds only when destination order equals receive order, which is true for the internal MPI_Scan permutation but not for a user permgsttab accepted by SCOTCH_dgraphRedist. E.g. a reversed 4-cycle sent to an empty rank: unpacking the third vertex overwrites data of the fourth. Release builds silently produce a wrong graph.

Fix: For user permutations, unpack into a separate edge array, or compute destination offsets by prefix sum and unpack in receive order.

Evidence: Code review, re-verified against source · wrong-result

DGR-05 **Medium** **dgraphBuild2 misses error flags from the upper half of the ranks**

`src/libscotch/dgraph_build.c:186-198, 212-217`

The Allgather stores (vertlocnbr, vertlocmax) pairs at procngbtab[2p] and [2p+1], but the error check reads procngbtab[procnum], so failures on ranks >= P/2 are missed; the others build procdsptab from -1 counts and hang in dgraphBuild3's collective.

Fix: Test procngbtab[2 * procnum] < 0.

Evidence: Code review, re-verified against source · mpi

DGR-06 **Medium** **dgraphInduce2 tests the prefix-sum array instead of the counts for remote errors**

`src/libscotch/dgraph_induce.c:181-186`

Failing ranks contribute -1 to proccnttab, but the loop tests procdsptab (the running prefix sum), which rarely goes negative. Remote errors are missed; the failing rank returns while others continue into later collectives (hang).

Fix: Test proccnttab[procngbnum] < 0.

Evidence: Code review, re-verified against source · mpi

DGR-07 **Medium** **Band/grow point-to-point error path frees an array owned by the caller**

`src/libscotch/dgraph_band_grow.c:394-399; dgraph_band.c:165-167; library_dgraph_grow.c:129-132`

On allocation failure, the Ptop variant frees the base-adjusted vnumgsttax, which the caller owns: band callers then free it again (double free), and in SCOTCH_dgraphGrow it is the user's partgsttab. Ptop also lacks Coll's vnumgsttax == NULL guard.

Fix: Remove memFree(vnumgsttax) and add the NULL check from the Coll variant.

Evidence: Code review · memory-safety

DGR-08 **Medium** **dgraphGhst2 keeps running after reporting a local allocation failure**

`src/libscotch/dgraph_ghst.c:140-157, 164, 290`

The failing rank performs the error reduction that matches the one others do at line 290 but never returns; it falls through to memSet on NULL arrays and then calls dgraphAllreduceMaxSum a second time (crash or unmatched collective).

Fix: return (1) after the reduction.

Evidence: Code review · mpi

DGR-09 **Medium** **dgraphBand out-of-memory path: NULL dereference, uninitialised free, overflow, wrong-base free**

`src/libscotch/dgraph_band.c:138, 156, 182-235, 250-256`

If the first allocation fails, procdsptab[0] = -1 dereferences NULL; later failures free the uninitialised bandfronloctab; the dummy Allgather writes procglbnbr GNUM_MPI values into a buffer sized with sizeof(int) (overflow with 64-bit Gnum); and bandvnumgsttax is freed with bandgrafptr->baseval after dgraphFree reset it to 0. Also reported by the public-API review.

Fix: Initialise work pointers to NULL, size the scratch buffer with sizeof(Gnum), and free with grafptr->baseval.

Evidence: Code review · memory-safety

DGR-10 **Medium** **dgraphGatherAll2 error path dereferences NULL and frees an uninitialised pointer**

`src/libscotch/dgraph_gather_all.c:154-217`

Non-root ranks have cgrfptr == NULL (dgraphGather case), but the error branch reads cgrfptr->verttax unconditionally; on the root, recvcnttab may be freed without being assigned.

Fix: Initialise recvcnttab = NULL and guard the block with cgrfptr != NULL.

Evidence: Code review · memory-safety

DGR-11 **Medium** **dgraphCheck never validates edge-end ranges; ends inside numbering holes read out of bounds**

`src/libscotch/dgraph_check.c:295-323, 421-422, 459-468`

Edge ends outside every process range match no neighbour and are silently accepted by SCOTCH_dgraphCheck; ends falling in a numbering hole index past the received neighbour data. Edge loads are only checked when the ghost edge array exists.

Fix: Bound lookups by the neighbour's vertex count, report unmatched ends, and check edge loads unconditionally.

Evidence: Code review · input-validation

DGR-12 **Medium** **dgraphBuild3 label translation assumes a compact edge array**

`src/libscotch/dgraph_build.c:365-368, 403-408, 425-433`

With vertex labels, edges to translate are taken from slots baseval..baseval+edgelocnbr-1. For non-compact graphs (allowed by the docs), gap slots are translated and real edges beyond edgelocnbr keep raw labels; labels found on no process are left untranslated without error.

Fix: Iterate over [vertloctax[v], vendloctax[v]) per vertex and error on untranslated entries.

Evidence: Code review · logic

DGR-13 **Medium** **Coarsening and matching return on local allocation failure before shared collectives (job hangs)**

`src/libscotch/dgraph_coarsen.c:604-607, 883-886, 1251-1254; dgraph_match_sync_coll.c:121-124`

These paths return immediately while other ranks enter dgraphAllreduceMaxSum or MPI_Alltoall(v); one rank running out of memory hangs the job (the code notes 'TODO: cleaner exit').

Fix: Record the error and join the next collective with an error marker, as dgraphGhst/dgraphFold do.

Evidence: Code review · mpi

DGR-14 **Low** **abs() applied to a pointer difference in dgraphRedist2 debug checks**

`src/libscotch/dgraph_redist.c:455, 464`

abs() takes int, but the argument is a ptrdiff_t (long); differences above 2^31 elements are truncated (gcc -Wabsolute-value), so the overlap assertion can give false results on large graphs.

Fix: Use labs()/llabs() or compare the pointers directly.

Evidence: Sanitizer report during test suite · ub

DGR-15 **Low** **dmeshLoadAdm leaks prelvrttab**

`src/libscotch/dmesh_io_load.c:271, 370, 381; dmesh.c:106-109`

prelvrttab is stored in the mesh but only DMESHFREETABS is set; dmeshFree2 frees it only under DMESHFREEPRIV, so it leaks on every exit (and an older prelvrttab is overwritten).

Fix: Set DMESHFREETABS | DMESHFREEPRIV and free any previous array.

Evidence: Code review, re-verified against source · memory-leak

DGR-16 **Low** **dmeshLoadAdm frees an unbased pointer on the 'bad input (2)' path**

`src/libscotch/dmesh_io_load.c:318-324, 361-364`

The 'goto fail' at line 321 runs before eelmloctax is rebased (line 324), but the fail path frees eelmloctax + baseval. For baseval = 1 this frees a pointer 8 bytes into the block, corrupting the heap on a malformed .adm file.

Fix: Rebase eelmloctax before the element-count check, or free the unbased pointer on that path.

Evidence: Code review · memory-safety

DGR-17 **Low** **Collective band/grow debug bound ignores the grow multiplier**

`src/libscotch/dgraph_band_grow.c:237-239`

The send area is DGRAPHBANDGROWSMUL(procrcvtab) but the debug bound uses plain procrcvtab, so SCOTCH_dgraphGrow reports false 'internal error (1)' in debug builds once more than half the ghost slots are used.

Fix: Compare against vsnddsptab[...] + DGRAPHBANDGROWSMUL(procrcvtab[...]).

Evidence: Code review · debug-check

DGR-18 **Low** **dgraphBuildGrid3D double-frees if dgraphBuild2 fails**

`src/libscotch/dgraph_build_grid3d.c:476-483`

DGRAPHFREETABS is set (with =, wiping other flags) before the build call; if dgraphBuild3 fails after storing the pointers, the routine frees the arrays and SCOTCH_dgraphExit frees them again.

Fix: Set flags with |= only after a successful build.

Evidence: Code review · memory-safety

DGR-19 Low **dgraphLoadCent abort path leaves sends pending and sends an unmatched message**

`src/libscotch/dgraph_io_load.c:422-447, 521-535`

When a receiver answers TAGBAD, the root breaks out without waiting on outstanding Isends and later frees the buffers; the abort loop starts at the current procnum, so the failed rank gets an extra zero-byte TAGVERTLOCTAB message it never receives.

Fix: MPI_Waitall pending requests before breaking and start the abort loop at procnum + 1.

Evidence: Code review · mpi

DGR-20 Low **dmeshDgraphDual loops forever if the hash-table resize fails**

`src/libscotch/dmesh_dgraph.c:592-595 (also 266-272, 316-323, 431-436, 507-510)`

The resize's return value is ignored; on memRealloc failure helmnbr >= helmmax stays true and goto redo repeats forever. Other allocation failures return without freeing or joining the following MPI_Alltoall(v).

Fix: Check the result and fail collectively.

Evidence: Code review · error-handling

DGR-21 Low **dgraphMatchCheck skips the last vertex when baseval != 0**

`src/libscotch/dgraph_match_check.c:113-120`

The scan runs from baseval to vertlocnbr instead of vertlocnnd, hiding an unmatched last vertex with baseval = 1 (debug only).

Fix: Loop to vertlocnnd.

Evidence: Code review · debug-check

DGR-22 Low **Gnum message sizes narrowed to int MPI counts; unchecked Waitall in fold**

`src/libscotch/dgraph_fold.c:343-452; dgraph_redist.c:295-321`

With 64-bit Gnum, per-message vertex/edge counts are cast to int while receivers size buffers from Gnum; dgraphRedist2 sums totals in int. Messages over INT_MAX items are truncated, and the MPI_Waitall at line 452 is unchecked.

Fix: Reject counts above INT_MAX collectively (or split messages / use MPI-4 large counts) and check Waitall.

Evidence: Code review · overflow

Distributed mapping & ordering (PT-Scotch) 19 findings · 1 high, 7 medium, 8 low, 3 info

DMO-01 High **hdgraphFold2 sends the wrong vertex-load slice when a sender has several receivers**

`src/libscotch/hdgraph_fold.c:360-367 (outer variables 303-312)`

A regression from commit 2747e2b: vertsndbas/vertsndnbr moved to an outer scope, where the first loop leaves them holding the last communication's values. The vertex-index loop shadows them correctly, but the vertex-load loop uses the stale outer ones, so with commnbr > 1 (e.g. 3 ranks) every receiver gets the last receiver's slice while posting a receive of its own size. The result is wrong weights in the folded graph, MPI_ERR_TRUNCATE, or uninitialised loads in parallel ordering of weighted graphs. dgraph_fold.c is correct.

Fix: Send veloloctax + fldvertidxtab[commnum] with count fldcommdattab[commnum].vertnbr, as dgraph_fold.c:359 does.

Evidence: Code review, re-verified against source · wrong-result

DMO-02 Medium **Vertex-label gathers in distributed ordering output are not based**

`src/libscotch/dorder_io.c:143-147, 202-208; dorder_io_tree.c:116-134`

An unbased receive buffer of vnodglbnbr entries is gathered with the based procdsptab as displacements, so with baseval = 1 commGatherv writes baseval elements past the allocation; dorderSave then prints the previous vertex's label on every line (the first is uninitialised). The same bug was fixed in dmapping_io.c (commit 4290e22) but not here.

Fix: Base the receive buffer before the gather, as dmapSave does.

Evidence: Code review, re-verified against source · memory-safety

DMO-03 Medium **bdgraphBipartSq tests the local failure count and never tests the memory flag**

`src/libscotch/bdgraph_bipart_sq.c:186-244`

The check reads reduloctab[4] (0 or 1 locally) instead of reduglbtab[4]: a failure on one rank makes only that rank return while others enter MPI_Bcast/commScatterv (mismatched collectives); when all fail, the centralised Bgraph and MPI datatype/op leak. reduglbtab[5] (allocation failure) is never tested, so an OOM rank scatters into NULL. This method runs at the coarsest levels of the default SCOTCH_dgraphMap/Part strategy.

Fix: Test reduglbtab[4], and check reduglbtab[5] before the broadcast, freeing what was built.

Evidence: Code review · mpi

DMO-04 **Medium** **Diffusion methods hide out-of-memory errors; vdgraphSeparateDf then uses freed memory**

`src/libscotch/vdgraph_separate_df.c:128-149; bdgraph_bipart_df.c:131-147`

On allocation failure reduloctab[0] = -1 is combined by MPI_SUM with other ranks' non-negative degrees, so the error is usually invisible and the failing rank writes through NULL. In vdgraphSeparateDf, even when detected, the error branch frees the arrays and falls through without return (use-after-free or double free at line 147).

Fix: Reduce a separate error flag with MPI_MAX and return (1) in the error branch.

Evidence: Code review · memory-safety

DMO-05 **Medium** **vdgraphSeparateMlUncoarsen leaks its exchange buffers on the 'no valid partition' path**

`src/libscotch/vdgraph_separate_ml.c:399-405, 449-454`

The buffer group (about the size of the local fine graph) is allocated before the validity check; when every coarse separator has an empty part the function returns 1 without freeing it. This is a normal fallback path that can repeat at many nested-dissection levels.

Fix: Free vsndcnttab before returning, or allocate after the check (as bdgraphBipartMlUncoarsen does).

Evidence: Code review · memory-leak

DMO-06 **Medium** **Leaf-ordering failures are ignored and reported as success**

`src/libscotch/hdgraph_order_nd.c:325-328; hdgraph_order_st.c:206`

The leaf's hdgraphOrderSt result (and hdgraphOrderSi in the empty-strategy case) is discarded and 0 returned. SCOTCH_dgraphOrderCompute reports success, but dorderPerm later fails with 'invalid parameters (2)' and dorderSave can wait forever for the missing fragment.

Fix: Propagate the results.

Evidence: Code review · error-handling

DMO-07 **Medium** **dorderNew result is not checked for NULL in nested dissection**

`src/libscotch/hdgraph_order_nd.c:362-364`

dorderNew is collective and returns NULL on failure, but the separator path dereferences the result immediately, crashing every rank (other call sites check).

Fix: if (cblkptr2 == NULL) goto fail;

Evidence: Code review · crash

DMO-08 **Medium** **hdgraphGather frees uninitialised pointers when an allocation on the root fails**

`src/libscotch/hdgraph_gather.c:94, 146-211`

verthaltax is assigned only inside if (cheklocval == 0), so the error path frees a garbage stack value; hgraphExit(cgrfptr) then runs on a Hgraph whose baseval and edgetax were never set.

Fix: Initialise verthaltax = NULL and the Hgraph fields before the first allocation.

Evidence: Code review · memory-safety

DMO-09 **Low** **The 'bal' strategy variable is stale or uninitialised after ml, sequential and zero bipartitioning**

`src/libscotch/bdgraph_bipart_ml.c:123-126, 753; bdgraph_bipart_sq.c:235-240; bdgraph.c:178-193; bdgraph_gather_all.c:248-254`

Uncoarsening copies bbalglbval from the rank's own coarse graph (not necessarily the selected one with duplicated folding); q and z methods never set it; bdgraphGatherAll never sets bbalval. A user strategy testing /(bal>...)? can branch differently on different ranks, which then run different collectives.

Fix: Recompute from compglbload0dlt / compglbload0avg wherever the partition changes.

Evidence: Code review · logic

DMO-10 **Low** **abs() applied to Gnum values in distributed bipartitioning**

`src/libscotch/bdgraph_bipart_ex.c:130, 245; bdgraph_bipart_st.c:355, 361-362; vdgraph_separate_st.c:320`

With 64-bit Gnum, abs truncates to int, so imbalances beyond 2^31 are miscomputed: bdgraphBipartEx can conclude there is nothing to do, and '|' strategy selection can pick the worse result. common.h already defines ABS.

Fix: Use ABS().

Evidence: Code review · overflow

DMO-11 Low **Parallel 'best partition' reductions compare signed imbalance**

`bdgraph_bipart_ml.c:208-210; bdgraph_bipart_sq.c:111-114; vdgraph_separate_ml.c:180-183; vdgraph_separate_sq.c:104-107`

The tie-breaker compares the signed load difference, so -500 beats +3; the sequential selection compares absolute values.

Fix: Reduce on the absolute value of the difference.

Evidence: Code review · logic

DMO-12 Low **vdgraphStoreUpdt does not restore local part loads**

`src/libscotch/vdgraph_store.c:117-155; vdgraph_separate_bd.c:109-110; vdgraph_check.c:230-236`

VdgraphStore saves global loads and local sizes but not complocload[]; after a '|' selection restores a partition, vdgraphSeparateBd passes stale local loads to dgraphBand, giving wrong (possibly negative) anchor loads. vdgraphCheck never compares local loads, so debug builds miss it.

Fix: Save/restore complocload[0..2] and check them in vdgraphCheck.

Evidence: Code review · logic

DMO-13 Low **dorderSaveBlock2 always leaks rangtab and hides write errors**

`src/libscotch/dorder_io_block.c:88-120`

rangtab is never freed, and unconditional trailing intSave calls overwrite the loop's error status, so I/O failures can be reported as success.

Fix: Free rangtab on every return and guard the trailing writes.

Evidence: Code review · memory-leak

DMO-14 Low **SCOTCH_dgraphOrderCblkDist returns 1 on error instead of a negative value**

`src/libscotch/dorder_tree_dist.c:119-120; library_dgraph_order_tree_dist.c:76`

Documented to return a negative value on error, it returns 1, which looks like a valid block count; callers sizing arrays from it overflow them in SCOTCH_dgraphOrderTreeDist.

Fix: Return -1.

Evidence: Code review · api-conformance

DMO-15 Low **A dmapSave validation check can never fire**

`src/libscotch/dmapping_io.c:123-126`

(reduglbtab[1] < 0) && (reduglbtab[1] > procglbnbr) is always false and uses the wrong bound, so inconsistent mappings are saved.

Fix: Use || and compare with vertglbnbr.

Evidence: Code review · input-validation

DMO-16 Low **Error-path leaks in hdgraph fold, nested dissection and kdgraph recursive bipartitioning**

`hdgraph_fold.c:321-325, 899-922; hdgraph_order_nd.c:370-374, 410-411; kdgraph_map_rb_part.c:373-398`

fail2/fail3 -> fail4 in hdgraphFold2 skips freeing fldcommdattab and the request group with sends pending; nested dissection leaks induced/folded graphs on failure; kdgraphMapRbPart2 returns from several failure points without freeing its graphs.

Fix: Route failures through cleanup labels that free owned buffers and complete pending requests.

Evidence: Code review · memory-leak

DMO-17 Info **Parallel mapping onto non-complete-graph targets is not implemented**

`src/libscotch/kdgraph_map_rb.c:260-262; kdgraph_map_rb_map.c:77-78`

Only cmplt/cmpltw/cmpltws/varcmplt take the kdgraphMapRbPart path; tleaf, mesh, torus, hcub, deco, sub etc. reach a stub that prints 'not implemented yet' and returns 1, so SCOTCH_dgraphMap always fails on them (consistently, without hanging).

Fix: Reject unsupported targets early with a clear message, or gather and map sequentially.

Evidence: Code review · missing-feature

DMO-18 Info **Copy-paste errors in distributed debug-only code**

`hdgraph_fold.c:891-893; bdgraph_bipart_ml.c:103-107; vdgraph_separate_ml.c:383, 450; vdgraph_separate_sq.c:181-185; hdgraph_order_st.c:190-191`

Wrong debug macros and function names (hdgraphFold2 prints 'dgraphFold2'), vdgraphSeparateSq sums an already-reduced value so its check always passes, and hdgraphOrderSt validates with && where siblings use ||.

Fix: Use each module's own macros and names, reduce the local flag, use ||.

Evidence: Code review · debug-check

DMO-19 Info **Unreachable external-gain code has indexing and accounting errors**

`bdgraph_bipart_ml.c:147-151; bdgraph_bipart_ex.c:465, 482, 491; bdgraph.c:98`

veexloctax is always NULL today, but the coarsening merge indexes it with global vertex numbers and the exact method adds twice the external-load change to commglbload; both break once external gains are enabled.

Fix: Convert to local numbers before indexing and add reduglbtab[4] / 2.

Evidence: Code review · latent

CLI tools, esMUMPS & METIS/ParMETIS shim

24 findings · 3 high, 9 medium, 12 low, 0 info

CLI-01 High **dgmap/dgpart silently discard the user's -m strategy**

`src/scotch/dgmap.c:257-263, 388-394`

-m parses the user strategy into stradat, but SCOTCH_stratDgraphMapBuild/ClusterBuild is then called unconditionally; both end in SCOTCH_stratDgraphMap, which frees the existing strategy first. Every -m strategy is replaced by the default, and the '-b/-c vs -m are exclusive' check is meaningless.

Fix: Wrap both Build calls in if (straptr == NULL), as dgord.c:308-313 does.

Evidence: Code review, re-verified against source · wrong-result

CLI-02 High **METIS_V5_PartGraphRecursive crashes when ubvec is NULL**

`src/libscotchmetis/metis_graph_part.c:587-595, 290`

METIS 5 documents ubvec as optional. The Kway wrapper substitutes a default for NULL; the Recursive wrapper passes ubvec through and _SCOTCH_METIS_PartGraph2 evaluates *kbalval, which segfaults on the most common call form.

Fix: Use the same (ubvec != NULL) ? ubvec : &kbalval fallback in both branches.

Evidence: Code review, re-verified against source · crash

CLI-03 High **METIS 5 ubvec (1+eps) is used unconverted as Scotch's imbalance fraction**

`src/libscotchmetis/metis_graph_part.c:553-562, 587-595, 290`

METIS ubvec[i] is a ratio such as 1.05; Scotch's balance parameter is a deviation applied as 1 +/- kbalval (kgraph_map_rb.c:140-141). Passing 1.05 allows about 105% imbalance instead of 5%, which badly degrades partitions for every METIS-API caller that sets ubvec.

Fix: Convert with kbalval = max(ubvec[0] - 1.0, 0) before building the strategy.

Evidence: Code review, re-verified against source · wrong-result

CLI-04 Medium **adm2dgr keeps running after errors (CMake build) and leaks or mis-frees arrays**

`src/libscotchmetis/adm2dgr.c:184, 201-205, 220; src/libscotchmetis/CMakeLists.txt:228`

CMake links ptscotcherr (errorPrint returns) while the Makefile links scotcherrexit, so in CMake builds a failed Mesh2Dual is followed by vertloctab[vertlocnbr] on an uninitialised pointer. prelvrttab is memAlloc'ed but released with free, and the dual arrays are never freed. The error message also names 'dmeshBuildAdm' and dgraphInit is given MPI_COMM_WORLD instead of proccomm.

Fix: Link the -exit error library in both build systems (or MPI_Abort), use memFree, and free the dual arrays.

Evidence: Code review, re-verified against source · error-handling

CLI-05 Medium **METIS_PartMeshDual reads out of bounds through Fortran-'null' vwgt/vsize**

`src/libscotchmetis/metis_graph_part_dual.c:206, 217-219`

Arrays equal to ne are treated as NULL elsewhere, but vwgt is passed to SCOTCH_graphBuild without that check (the scalar ne is read as an array), and vsize - baseval is passed even when vsize is NULL, so NULL - 1 reaches _SCOTCH_METIS_OutputVol. The graphBuild result is ignored too.

Fix: Normalise vwgt/vsize equal to ne to NULL, offset vsize only when non-NULL, and check graphBuild.

Evidence: Code review · memory-safety

CLI-06 Medium **METIS V5 wrappers use options[NUMBERING] = -1 ('default') as the base value**

`src/libscotchmetis/metis_graph_part.c:554, 587; metis_graph_part_dual.c:108-109; metis_options.c:70`

METIS 5 uses -1 to mean 'default' in options[]; many callers fill the array with -1. The wrappers use the slot directly as baseval, so baseval = -1 breaks graph building and output adjustment. METIS_SetDefaultOptions zero-fills, hiding this.

Fix: Treat negative NUMBERING as 0, and reject values other than 0/1 with METIS_ERROR_INPUT.

Evidence: Code review · api-conformance

CLI-07 **Medium** **Multi-constraint (ncon > 1) weights are misread; ParMETIS ignores the tpwgts stride and ubvec**

`src/libscotchmetis/metis_graph_part.c:275, 287-288; parmetis_dgraph_part.c:156, 169`

With ncon > 1, vwgt holds nvtxs*ncon interleaved weights but is used as one load per vertex, so vertex i receives constraint i%ncon of vertex i/ncon. ParMETIS converts tpwgts without striding by ncon, mixing constraints, and never uses ubvec.

Fix: Extract the first constraint into a temporary array (or return METIS_ERROR_INPUT), and stride tpwgts by ncon.

Evidence: Code review · api-conformance

CLI-08 **Medium** **tpwgts-to-integer conversion can blow up the scale factor (UB on cast)**

`src/libscotchmetis/metis_graph_part.c:216-229; parmetis_dgraph_part.c:79-110`

The common-denominator search multiplies doubadj by reciprocals of fractional residues; for arbitrary real weights it exceeds 1e13 within a few steps, making (int) round(doubadj) an out-of-range conversion (UB) and overflowing SCOTCH_Num weights. A Python replica reached about 2e13 for 6 random weights.

Fix: Scale to a fixed resolution (e.g. round(w * 1e6), minimum 1) or cap doubadj.

Evidence: Code review · overflow

CLI-09 **Medium** **ParMETIS_V3_NodeND returns METIS_ERROR on valid runs and METIS_OK on failed ones**

`src/libscotchmetis/parmetis_dgraph_order.c:150-157, 206-208`

o = METIS_OK is set only when the process count is a power of two and the separator tree has the expected shape; otherwise METIS_ERROR is returned even though order was filled. Meanwhile the return codes of SCOTCH_dgraphOrderCompute and OrderPerm are ignored.

Fix: Derive the status from the Compute/Perm results; treat filling sizes as best effort.

Evidence: Code review · error-handling

CLI-10 **Medium** **esmumps2 ignores every failure and returns success with garbage output**

`src/esmumps/esmumps.c:139-143, 171`

The results of graphBuildGraph2, orderGraph and symbolFaxGraph are discarded. After an out-of-memory failure the loop over symbdat.cblktab reads invalid structures and the function still returns 0, so MUMPS consumes corrupt pe/nv arrays.

Fix: Check each call, clean up what was built, and return a nonzero (MUMPS-style negative) code.

Evidence: Code review · error-handling

CLI-11 **Medium** **amk_grf -l passes 0-based vertex indices where graph-based ones are expected**

`src/scotch/amk_grf.c:206, 218; src/libscotch/arch_deco_build.c:301`

For labelled graphs, list labels become sorttab[].vertnum, which starts at 0, but archDecoArchBuild indexes with the graph base. Base-1 graphs are off by one and read index 0 out of bounds; for unlabelled graphs raw list values are never range-checked.

Fix: Store vertnum + baseval and reject values outside [baseval, baseval + vertnbr).

Evidence: Code review · off-by-one

CLI-12 **Medium** **gout Inventor and VTK outputs are wrong for base-1 graphs**

`src/scotch/gout_o.c:405, 1121, 1142, 1173-1176`

Inventor subtracts baseval from every coordIndex entry including the -1 polyline terminators (giving -2). VTK writes based vertex numbers into CELLS while POINTS is 0-indexed, so base-1 graphs get shifted cells and an out-of-range index.

Fix: Un-base only non-terminator entries in Inventor; write verttmp - baseval in VTK.

Evidence: Code review · wrong-result

CLI-13 **Low** **atst always reports a minimum distance of -1**

`src/scotch/atst.c:161`

The 'maximum value' initialiser (SCOTCH_Num)((unsigned long)(SCOTCH_Num)-1 >> 1) truncates to -1 on LP64 with 32-bit SCOTCH_Num, so 'if (distmin > distval)' never fires. Every architecture tested printed 'min=-1'.

Fix: Initialise with ((SCOTCH_Num)(((unsigned SCOTCH_Num) ~0) >> 1)) or a SCOTCH_NUMMAX constant.

Evidence: Reproduced by running code · logic

CLI-14 Low **gcv accepts any unknown output format letter**

`src/scotch/gcv.c:187`

The -o loop scans C_outFormatTab but tests C_inpFormatTab[j].code for the terminator; for an unknown code j stops at 6 where C_inpFormatTab[6] is 'S', so no error is raised and the previous/default format is used.

Fix: Test C_outFormatTab[j].code == '\0'.

Evidence: Code review, re-verified against source · cli

CLI-15 Low **Dual-graph helpers mishandle memory on error paths**

`src/libscotchmetis/metis_graph_dual.c:112-118; parmetis_dgraph_dual.c:150-171`

METIS_MeshToDual frees *xadj on OOM but leaves the dangling pointer in the caller's variable and skips graphExit. ParMETIS_V3_Mesh2Dual allocates with plain malloc but frees with memFree (breaks under COMMON_MEMORY_TRACE/CHECK), and its SCOTCH_DEBUG_ALL early return leaks both arrays.

Fix: NULL out the caller's pointers after freeing, pair malloc with free, and free on every early return.

Evidence: Code review, re-verified against source · memory-leak

CLI-16 Low **gout Tulip 'no geometry' check can never fire**

`src/scotch/gout_o.c:921-926`

coortax is computed as coortab - baseval before the NULL test, so with baseval = 1 and no geometry it is NULL - 1 (non-NULL); the error is skipped and coortax[vertnum] is dereferenced.

Fix: Test geomptr->coortab == NULL instead.

Evidence: Code review · crash

CLI-17 Low **gout option parser dereferences NULL on an unterminated option list**

`src/scotch/gout_c.c:708-710`

strpbrk returns NULL when neither ',' nor '}' is present; an option such as -Oi{c crashes on *argeptr instead of reporting a syntax error.

Fix: Check argeptr == NULL (or use strcspn).

Evidence: Code review · crash

CLI-18 Low **gmk_m2 -e -t writes a wrong edge count for 2x2 and degenerate tori**

`src/scotch/gmk_m2.c:197-200`

The closed-form arc count for the 8-neighbour torus does not match the arcs written when both dimensions are <= 2 (2x2: header 8, 12 written; 1x2: header -2). The produced files fail to load.

Fix: Count arcs by summing per-vertex degrees, or special-case small dimensions.

Evidence: Code review · wrong-result

CLI-19 Low **METIS V5 partitioning chooses the objective from vsize instead of OBJTYPE**

`src/libscotchmetis/metis_graph_part.c:556-562, 589-595`

The wrappers branch on vsize == NULL only: OBJTYPE=VOL without vsize returns edge cut, and the default OBJTYPE=CUT with vsize switches to volume. PartMeshDual does honour OBJTYPE.

Fix: Select the variant from options[OBJTYPE] (treating -1 as CUT), using unit sizes when vsize is NULL.

Evidence: Code review · api-conformance

CLI-20 Low **gmap/gpart help text contradicts the code**

`src/scotch/gmap.c:105, 170, 250`

Help says the default -b imbalance is 0.05, but kbalval defaults to 0.01; invalid -c characters are reported as 'after -C' (a different flag).

Fix: Make the help match the default and fix the message.

Evidence: Code review · docs

CLI-21 Low **PT-Scotch tools skip MPI_Finalize on early exits and never MPI_Abort on errors**

`src/scotch/dgmap.c:176-178, 256, 307; dgord.c:154-156, 223, 255; dgscat.c, dggath.c, dgtst.c; src/libscotch/library_error_exit.c:142`

The ?, -h and -V paths return after MPI_Init without MPI_Finalize. errorPrint calls exit() instead of MPI_Abort, so when one rank fails on a bad file the others can hang in the next collective.

Fix: Finalize before returning; in the PT error-exit library call MPI_Abort when MPI is initialised.

Evidence: Code review · mpi

CLI-22 Low **test_esmumps cannot detect esmumps failures**

`src/esmumps/test_esmumps.c:146-147; esmumps.c:134; library_esmumps_f.c:81`

esmumps2 returns +1 on error and that value is stored in ncmpa, but the test only fails when ncmpa < 0. It also prints a possibly 64-bit INT with %d.

Fix: Use one sign convention (negative on error, MUMPS-style) and print with INTSTRING.

Evidence: Code review · testing

CLI-23 Low **esmumps orderLoad/orderSave lack validation and lose write errors**

`src/esmumps/order_io.c:119-123, 166-176`

orderLoad never range-checks permtab values before peritax[permtax[v]] = v (out-of-bounds write on malformed files). orderSave overwrites the loop's error status with the final intSave, and reads permtab[0] when vertnbr == 0.

Fix: Validate counts and permutation entries; accumulate statuses; guard vertnbr == 0.

Evidence: Code review · input-validation

CLI-24 Low **Generators overflow 1 << dim for large dimensions**

`src/scotch/amk_hy.c:149-150; gmk_hy.c:146-147; gmk_ub2.c:164-165; amk_fft2.c:154-155; amk_ccc.c:157-158, 170`

Dimensions from atoi are only checked for < 1; shifts are on int 1, so dimension >= 31 is UB even with 64-bit SCOTCH_Num and corrupts counts and allocation sizes.

Fix: Bound the dimension and shift ((SCOTCH_Num) 1).

Evidence: Code review · overflow

Build system, tests & CI 25 findings · 4 high, 13 medium, 7 low, 1 info

BLD-01 High **CMake build silently omits five libscotch source files**

`src/libscotch/CMakeLists.txt:151 (SCOTCH_C_SOURCES); src/libscotch/Makefile:376, 431-432, 449, 470`

hgraph_dump.c, library_context_mesh.c, library_context_mesh_f.c, library_graph_induce_f.c and library_memory_f.c are built by the Makefile but listed in no CMakeLists.txt. A CMake-built libscotch has no SCOTCH_contextBindMesh (declared in scotch.h) and no Fortran wrappers for graph induce, memory or context-bind-mesh, so user code calling them fails to link (confirmed with nm). Also found independently by the public-API review.

Fix: Add the files to SCOTCH_C_SOURCES and add a CI check that diffs the Makefile object list against the CMake source list.

Evidence: Code review, re-verified against source · cmake

BLD-02 High **find_package(SCOTCH) fails on installs built with SCOTCH_NAME_SUFFIX**

`cmake/SCOTCHConfig.cmake.in:85-92; src/CMakeLists.txt:224`

esmumps (and its install(EXPORT esmumpsTargets)) is skipped when a suffix is used, but the installed config file includes esmumpsTargets.cmake whenever BUILD_LIBESMUMPS is ON (the default). include() has no OPTIONAL, so every consumer's find_package(SCOTCH) fails on a suffixed install.

Fix: Guard the include with @BUILD_LIBESMUMPS@ AND NOT @USE_SUFFIX@, or use include(... OPTIONAL).

Evidence: Code review, re-verified against source · cmake

BLD-03 High **Sonar login token is written to a file kept as a CI job artifact**

`ci/analysis.sh:26-28; .gitlab-ci.yml:133-136`

The unquoted heredoc expands $SONARQUBE_LOGIN into sonar-project.properties, which the scotch_sonar job archives for 15 days. This bypasses GitLab log masking; anyone able to download artifacts can read the token. The archived sonar-scanner -X logs may leak it too.

Fix: Pass the token via SONAR_TOKEN / -Dsonar.token, drop the file and -X logs from artifacts, and rotate the current token.

Evidence: Code review, re-verified against source · secrets

BLD-04 High **test_scotch_graph_order can never fail on a bad ordering, and can read out of bounds**

`src/check/test_scotch_graph_order.c:74, 94-104`

checkOrder only calls SCOTCH_errorPrint, and the test links the non-exiting scotcherr library, so a wrong permutation still passes. After an out-of-range permval, peritab[permval - baseval] is read out of bounds.

Fix: exit(EXIT_FAILURE) after each error (as test_libmetis.c does) and skip the pair check when permval is out of range.

Evidence: Code review, re-verified against source · test-gap

BLD-05 **Medium** **CMake gpart_overlap_2 checks nothing: gmtst runs without -o**

`src/check/CMakeLists.txt:685-686; src/check/Makefile:776`

The Makefile runs gmtst -o on the overlap map; CMake omits -o. A CTest log shows the test passing while gmtst prints meaningless 'Processors 0/32' statistics.

Fix: Add -o to match the Makefile.

Evidence: Code review · test-gap

BLD-06 **Medium** **Open MPI detection regex never matches, so --oversubscribe is never added**

`src/CMakeLists.txt:181-185`

The pattern requires a space before 'Open MPI', but the version string starts with it ('Open MPI v4.1.2, ...'). Reproduced with cmake -P. On hosts with fewer than 4 cores the 4-process MPI tests fail with not-enough-slots (also hit in this review's own test run).

Fix: Use if(MPI_C_LIBRARY_VERSION_STRING MATCHES "Open MPI").

Evidence: Code review · cmake

BLD-07 **Medium** **Makefile CI build step cannot fail (pipe into tee without pipefail)**

`ci/build_makefile.sh:19`

make ... | tee log || fatal takes tee's exit status, so every scotch_build_* job goes green when compilation fails; the failure surfaces later in the test stage.

Fix: set -o pipefail, or check ${PIPESTATUS[0]}.

Evidence: Code review · ci

BLD-08 **Medium** **CMake test_dgscat* hides scatter and gather failures**

`src/check/CMakeLists.txt:739-758`

dgscat, dggath and compare_files are joined with ';', so only the final comparison counts; a stale out_dggat.grf from an earlier run makes the test pass when both MPI programs fail.

Fix: Join with && and delete outputs before running.

Evidence: Code review · test-gap

BLD-09 **Medium** **scotchf.h / ptscotchf.h depend on the wrong template header**

`src/libscotch/CMakeLists.txt:77-80, 100-103`

Both custom commands depend on library.h instead of library_f.h / library_pt_f.h, so editing a Fortran template does not regenerate the installed Fortran header.

Fix: DEPENDS library_f.h / library_pt_f.h.

Evidence: Code review · cmake

BLD-10 **Medium** **suffix_fortran_file uses an undefined ${file}, so .f90.in edits are not tracked**

`src/check/CMakeLists.txt:68-80, 100-101`

The parameter is src; ${file} is empty, so the dependency resolves to the source directory. After the first build, editing a Fortran test template never regenerates the compiled .f90.

Fix: Depend on ${CMAKE_CURRENT_SOURCE_DIR}/${src} and on the .tmp file path.

Evidence: Code review · cmake

BLD-11 **Medium** **parmetis_dgraph_dual_f object lacks its parmetis.h dependency**

`src/libscotchmetis/Makefile:205-208`

The source includes generated parmetis.h, but the rule does not list it, so under make -j it can compile before the header exists and is not rebuilt when it changes.

Fix: Add parmetis.h to the prerequisites.

Evidence: Code review · makefile

BLD-12 **Medium** **Generated metis.h is shared by independent targets (parallel-build race)**

`src/libscotchmetis/CMakeLists.txt:56-62, 89, 111, 152-172`

metis.h is a source of both scotchmetisv3 and scotchmetisv5, neither of which depends on the metis_h target; with the Makefile generator both can run the rule and write the file concurrently.

Fix: add_dependencies on metis_h / parmetis_h and drop the generated header from per-target sources.

Evidence: Code review · cmake

BLD-13 **Medium** **cmake_minimum_required(3.10) is lower than the features used**

`CMakeLists.txt:44; src/check/CMakeLists.txt:69, 289; README.md:70`

NAME_WLE needs CMake 3.14 and cmake -E cat needs 3.18, while README and INSTALL promise 3.10, so configuration or testing fails on 3.10-3.17.

Fix: Raise the minimum to 3.18 (and update docs) or use 3.10-compatible constructs.

Evidence: Code review · portability

BLD-14 **Medium** **CI never tests 64-bit SCOTCH_Num on 64-bit hosts, or non-default CMake options**

`ci/env_makefile.sh:7-14; .gitlab-ci.yml:34-48; ci/build_cmake.sh:11`

JOB_INT=64 adds no flags, so '64' jobs build 32-bit integers; -DINTSIZE64 is only exercised on 32-bit runners. CMake jobs never set INTSIZE, IDXSIZE, SCOTCH_NAME_SUFFIX or SCOTCH_DETERMINISTIC, which is how the suffix config breakage went unnoticed.

Fix: Make '64' add -DINTSIZE64 and add a CMake matrix entry with INTSIZE=64 and a suffix.

Evidence: Code review · ci

BLD-15 **Medium** **test_scotch_mesh_graph passes when the converted graph is invalid**

`src/check/test_scotch_mesh_graph.c:113-118`

A failing SCOTCH_graphCheck only prints a message (non-exiting scotcherr) and the program exits with success.

Fix: exit(EXIT_FAILURE) inside the if.

Evidence: Code review · test-gap

BLD-16 **Medium** **test_scotch_graph_map accepts unmapped vertices and never checks fixed vertices**

`src/check/test_scotch_graph_map.c:95-106`

checkMap skips -1 entries, so an all-unmapped result passes; with flagval set there is no upper bound check, and fixed vertices are never verified to keep their parts.

Fix: Reject -1 for non-fixed runs, compare fixed vertices to a saved copy, and bound partval by the architecture size.

Evidence: Code review · test-gap

BLD-17 **Medium** **make -j: esmumps and ptesmumps sub-makes can race in the same directory**

`src/Makefile:122-129; src/esmumps/Makefile:61`

ptesmumps does not depend on esmumps and its ptscotch goal rebuilds the scotch goal, so 'make -j esmumps ptesmumps' (what CI runs) can compile the same objects and archive libesmumps.a concurrently.

Fix: Make ptesmumps depend on esmumps and only run ptinstall.

Evidence: Code review · makefile

BLD-18 **Low** **make check never runs the mcv tests**

`src/check/Makefile:199-241, 702`

check_prog_mcv is defined but missing from CHECKSCOTCH, so mesh conversion regressions are only caught by CMake.

Fix: Add check_prog_mcv to CHECKSCOTCH.

Evidence: Code review · makefile

BLD-19 **Low** **The esmumpsv test path is dead code**

`src/check/test_libesmumps.c:181-196`

ESMUMPS_HAS_ESMUMPSV is defined nowhere, so the public esmumpsv() routine is never tested.

Fix: Remove the guard or define the macro in both build systems.

Evidence: Code review · test-gap

BLD-20 **Low** **Fortran METIS_PartMeshDual test checks no results**

`src/check/test_libmetis_dual_f.f90.in:134-138`

objval, epart and npart are never range-checked; the test only fails on a crash, unlike its C counterpart.

Fix: Check every entry is in [baseval, baseval + nparts) and stop 1 otherwise.

Evidence: Code review · test-gap

BLD-21 Low **Make.inc templates set macros the code never tests**

`src/Make.inc/Makefile.inc.i686_pc_mingw32:29; Makefile.inc.*_pc_haiku.shlib:13`

MinGW sets -DCOMMON_WINDOWS (code uses COMMON_OS_WINDOWS), disabling the Windows code paths; Haiku sets -DCOMMON_FILE_COMPRESS_BZ (code tests ..._BZ2), silently disabling bzip2.

Fix: Rename to COMMON_OS_WINDOWS and COMMON_FILE_COMPRESS_BZ2.

Evidence: Code review · portability

BLD-22 Low **Man pages omit implemented options**

`man/man1/gmap.1.txt:45-75; gord.1.txt:27-53; dgord.1.txt:32-60`

gmap/gpart -C, -f, -o, -ro/-rr/-rv and -s are missing from gmap.1; gord.1 lacks -C; dgord.1 lacks -b and -C.

Fix: Document the options and regenerate the man pages.

Evidence: Code review · docs

BLD-23 Low **gscat usage shows -i without its required argument**

`src/scotch/gscat.c:73, 121-124`

Help shows a bare -i but the parser requires -i<num> >= 1; typing -i as documented fails.

Fix: Change the usage line to -i<nbr>.

Evidence: Code review · docs

BLD-24 Low **Internal compile definitions leak to consumers of the exported scotch target**

`src/libscotch/CMakeLists.txt:542, 546-547, 550`

COMMON_RANDOM_FIXED_SEED, SCOTCH_DETERMINISTIC, cpu_set_t=cpuset_t and __BSD_VISIBLE are PUBLIC, so downstream projects compile with them (on FreeBSD a type-rename macro and changed header visibility).

Fix: Make them PRIVATE.

Evidence: Code review · cmake

BLD-25 Info **Typos in src/CMakeLists.txt option handling**

`src/CMakeLists.txt:67, 131`

The FreeBSD test has a stray '}' and works by accident; the IDXSIZE validation error says 'integer size' (copied from INTSIZE).

Fix: Fix the condition and the message.

Evidence: Code review · cmake

Generated from the same dataset as the Cursor canvas scotch-bug-review.canvas.tsx. The repository was not modified during this review.